



***Asociación Profesional del Cuerpo Superior
de Sistemas y Tecnologías de la Información
de la Administración del Estado***

Temas Específicos para la preparación de la Oposición al Cuerpo
Superior de Sistemas y Tecnologías de la Información de la
Administración del Estado.

TEMAS ESPECÍFICOS III: Ingeniería de los Sistemas de Información

**Tema 91. Análisis de aspectos no funcionales: Rendimiento,
seguridad, privacidad**

AUTOR: José Antonio Eusamio Mazagatos

Actualización 2017

ÍNDICE

1	INTRODUCCIÓN	3
1.1	CARACTERÍSTICAS DEL DESARROLLO Y DE LOS REQUISITOS SOFTWARE	3
1.2	CLASIFICACIÓN DE REQUISITOS NO FUNCIONALES	5
1.2.1	IMPORTANCIA DE CLASIFICAR LOS REQUISITOS NO FUNCIONALES	6
1.2.2	REQUISITOS NO FUNCIONALES DE PRODUCTO	8
1.2.3	REQUISITOS NO FUNCIONALES ORGANIZACIONALES	9
1.2.4	REQUISITOS NO FUNCIONALES EXTERNOS	9
1.2.5	IMPORTANCIA DE CLASIFICAR LOS REQUISITOS NO FUNCIONALES	10
1.2.6	FURPS	10
1.3	RENDIMIENTO - PERFORMANCE	11
1.3.1	LA OPTIMIZACIÓN DEL RENDIMIENTO (TUNNING)	11
1.3.2	RENDIMIENTO PERCIBIDO.....	12
1.4	SEGURIDAD.	12
1.4.1	MEDIDAS DE PREVENCIÓN CONTRA LAS VULNERABILIDADES.....	12
1.4.2	INTERFAZ DE SEGURIDAD EN MÉTRICA	13
1.5	PRIVACIDAD.....	15
1.5.1	PARADOJA DE LA PRIVACIDAD	15
2	CALIDAD DE REQUISITOS NO FUNCIONALES: MEDIDAS 17	
2.1	MÉTRICAS, MEDIDAS Y CALIDAD DE REQUISITOS NO FUNCIONALES.....	17
2.1.1	COMPONENTES DE UN MODELO DE CALIDAD	18
2.1.2	MEDIDAS DE RENDIMIENTO - PERFORMANCE.....	19
2.1.3	MEDIDAS DE FIABILIDAD	20
2.1.4	MEDIDAS DE DISPONIBILIDAD.....	20
2.1.5	MEDIDAS DE USABILIDAD	23
2.1.6	MEDIDAS DE MANTENIMIENTO.....	23
2.1.7	MEDIDAS DE TESTABILIDAD	24
2.1.8	MEDIDAS DE PORTABILIDAD	24
2.1.9	MEDIDAS DE INTEGRABILIDAD Y REUTILIZACIÓN.....	24
2.1.10	MEDIDAS DE ROBUSTEZ	25
2.1.11	MEDIDAS DE SEGURIDAD	25
2.1.12	MEDIDAS DE PRIVACIDAD.....	26
2.1.13	OTROS REQUISITOS NO FUNCIONALES	26
3	ESTÁNDARES RELATIVOS A REQUISITOS.....	28
3.1	ISO/IEC 25010.....	28
3.1.1	ADECUACIÓN FUNCIONAL	29
3.1.2	EFICIENCIA DE DESEMPEÑO	29
3.1.3	COMPATIBILIDAD.....	30
3.1.4	USABILIDAD	30
3.1.5	FIABILIDAD	30
3.1.6	SEGURIDAD	30
3.1.7	MANTENIBILIDAD.....	31
3.1.8	PORTABILIDAD	31
4	EJEMPLO DE DOCUMENTO DE REQUISITOS NF - SGAD .	32
5	GLOSARIO	36

1 Introducción

El desarrollo del software ha ido pasando por diversas etapas. En el principio de los tiempos de la computación, hacia los años 40 del siglo XX, el desarrollo software era completamente artesanal, cada usuario se hacía su propio software. Con el enorme avance que fue experimentando la informática, en poco tiempo se pasó de hacerse los usuarios su propio software a constituirse una profesión específica, la de programador, el cual se encargaba de construir a otros el software que estos necesitaban. A la vez, cada poco tiempo la potencia de cálculo de los ordenadores se duplicaba, con lo que cada vez era posible hacer más tareas con ellos, lo cual por otro lado también complicaba más el software.

Este aumento de complejidad llevó a lo que se denominó “Crisis del Software” en el año 1968 en una conferencia organizada por la Comisión de Ciencias de la OTAN y celebrada en Garnisch (Alemania). Por crisis del software se entiende al “conjunto de problemas que se han ido encontrando en el desarrollo software”. Esta problemática no sólo se limita al software que no funciona adecuadamente, sino que abarca otros aspectos como:

- ✓ La forma de desarrollar el software.
- ✓ El mantenimiento de un volumen creciente de software existente.
- ✓ La forma de satisfacer la demanda creciente de software.

Los **síntomas** que hicieron palpable la aparición de la “*crisis del software*” son, entre otros, los siguientes:

- ✓ Expectativas: Los sistemas no responden a las expectativas que de ellos tienen los usuarios.
- ✓ Fiabilidad: Los programas fallan demasiado a menudo.
- ✓ Coste: Los costes del software son muy difíciles de prever y, frecuentemente, son muy superiores a lo esperado.
- ✓ Plazos: El software se suele entregar tarde y con menos prestaciones de las ofertadas.
- ✓ Portabilidad: Es difícil cambiar un programa de su entorno hardware, aun cuando las tareas a realizar sean las mismas.
- ✓ Mantenimiento: La modificación del software es costosa, compleja y propensa a errores.
- ✓ Eficiencia: Los esfuerzos que se hacen para el desarrollo del software no hacen un aprovechamiento óptimo de los recursos disponibles (personas, tiempo, dinero, herramientas, etc.).

A finales de los 60 surge el concepto de “Ingeniería del Software” para garantizar que un proyecto se realizaba en tiempo y coste, y que además satisfacía las expectativas de los usuarios.

Estas expectativas se derivan de lo que denominamos requisitos del sistema. Los requisitos del sistema se clasifican en “Funcionales” y “No Funcionales”

- ✓ Los requisitos “Funcionales” son aquellos derivados de la necesidad del “Negocio”, es decir lo que se tiene que hacer.
- ✓ Los requisitos “No Funcionales” son aquellos derivados de la Manera en la que se tienen que satisfacer los requisitos del “Negocio”, es decir el COMO se tiene que hacer. No se refieren directamente a las funciones específicas que entrega el sistema, sino a las propiedades emergentes de éste como la fiabilidad, la respuesta en el tiempo y la capacidad de almacenamiento.

Existen distintas prácticas para la identificación de requisitos (tanto Funcionales como No Funcionales), si bien suelen ser parecidas y se explican en el tema relativo al análisis de los Requisitos Funcionales.

El objetivo de este tema es profundizar en los aspectos “No funcionales” del sistema, que pueden llegar a ser tan importante como los funcionales.

1.1 Características del Desarrollo y de los requisitos Software

Hay que tener en cuenta que el software es diferente al resto de productos que se fabrican. Éste tiene algunas características que le hacen especial:

- El software no se fabrica, **se desarrolla**.
- Los costes se concentran en **la ingeniería**, no en la producción.
- El software **no se estropea**, pero se “deteriora”.

Debido a sus características especiales, éste también presenta sus problemas específicos a la hora de desarrollarse:

- Es intangible, por lo tanto es más difícil estimar los costes de desarrollo.
- Es muy dependiente de la tecnología y del estado del arte

El proceso de construcción no está tampoco exento de problemas. Algunos de éstos son los siguientes:

- Difícil extraer, comprender y reflejar requisitos.
- Difícil de automatizar el proceso de construcción.
- Difícil de probar.
- Difícil de modificar. Cambios sin comprender su alcance e impacto.

Para entender y comprender la importancia de los requisitos resumimos las **Características que deben cumplir todos los requisitos, tanto Funcionales como no funcionales. Un requisito debe ser:**

- ✓ **Correcto**. Cada requisito establecido debe representar algo requerido o solicitado
- ✓ **No Ambiguo**. Cada requisito establecido tiene una sola interpretación.
 - Todos los términos con múltiples significados deberán aparecer en un glosario que permita la desambiguación.
- ✓ **Completo**. Debe incluir todo lo que el software tiene que hacer.
- ✓ **Verificable**. Se ha de poder comprobar, mediante un proceso efectivo y de coste limitado, que el producto reúne cada requisito establecido.
- ✓ **Consistente**. Cada requisito no puede estar en conflicto con otros requisitos.
- ✓ **Modificable**. La estructura y estilo del documento debe hacer fáciles los cambios.
- ✓ **Conciso y Comprensible por el usuario**. No sólo por el desarrollador.
- ✓ **Independiente del diseño**. El documento no implica una arquitectura software o un diseño.
- ✓ **Organizado y Referenciado**. Fáciles de encontrar, calificado y debidamente referenciado.

Un requisito no funcional, en adelante RNF, es un requisito que especifica criterios que pueden usarse para juzgar el funcionamiento de un sistema en referencia a comportamientos específicos. Se contrastan con los requisitos funcionales que definen comportamientos o funciones específicas.

- ✓ El plan para implementar requisitos funcionales se detalla en el diseño del sistema.
- ✓ El plan para la implementación de requisitos no funcionales se detalla en la arquitectura del sistema, ya que suelen ser Requisitos Arquitectónicamente significativos.

Los requisitos funcionales suelen expresarse en forma de **"sistema debe hacer <requisito>"**, una acción individual de parte del sistema, tal vez explícitamente en el sentido de una función matemática, una caja negra de descripción de entrada, salida, proceso y modelo funcional de control o Modelo IPO. Por el contrario, los requisitos no funcionales están en forma de **"sistema será / ejecutará / conseguirá <requisito>"**, una propiedad general del sistema en su conjunto o de un aspecto particular y no una función específica. Las propiedades generales del sistema marcan generalmente la diferencia entre si el proyecto de desarrollo ha tenido éxito o ha fallado.

Los requisitos no funcionales a menudo se llaman **"atributos de calidad"** de un sistema. Otros términos para requisitos no funcionales son **"cualidades"**, **"objetivos de calidad"**, **"requisitos de calidad de servicio"**, **"limitaciones"** y **"requisitos no relacionados con el comportamiento"**. Informalmente, en Inglés se refiere a ellos como **"-ilities"** como la estabilidad (stability) o la portabilidad (portability).

Algunos ejemplos de requisitos no funcionales son:

- ✓ **Adaptabilidad**: a otros entornos o problemas

- ✓ **Complejidad:** grado de dependencia e interacción entre Componentes
- ✓ **Comprensibilidad:** de diseño, arquitectura, código
- ✓ **Comprobabilidad:** Grado en que un sistema, software o servicio de TI permite y facilita que sea probado en un determinado contexto.
- ✓ **Confiabilidad:** de cálculos, precisión
- ✓ **Coste:** coste total de propiedad (TCO) para adquisición, instalación, uso, disposición
- ✓ **Disponibilidad:** Corresponde al tiempo total en que un sistema puede ser usado en un período determinado. También puede definirse el grado en que un sistema está en un estado operable definido cada vez que se necesite.
- ✓ **Eficiencia:** uso mínimo de recursos (memoria, procesador, disco, red...)
- ✓ **Escalabilidad:** Capacidad de un sistema o servicio de TI de manejar una creciente carga de trabajo, por ejemplo mayor número de conexiones o usuarios o cantidades de datos o incrementos de las cifras de rendimiento actuales. La escalabilidad puede ser horizontal (más Equipos) o vertical (más capacidad de los equipos). No debe confundirse con extensibilidad, que mide la capacidad del sistema de crecer en funcionalidades.
- ✓ **Extensibilidad:** Grado en que la implementación del sistema toma en consideración y facilita su crecimiento en el futuro.
- ✓ **Integrabilidad:** capacidad para integrar componentes
- ✓ **Legibilidad:** de código, documentos
- ✓ **Mantenibilidad:** Mide la facilidad con que puede darse mantenimiento al producto (en este caso al software o servicio de TI), con la finalidad de: desarrollar nuevos requisitos/funcionalidades, aislar los defectos y sus causas, corregir estos defectos y atender las demandas del entorno cambiante.
- ✓ **Modificabilidad:** adición de nuevas funcionalidades
- ✓ **Portabilidad:** Funcionamiento sobre varias plataformas
- ✓ **Privacidad:** Ámbito de la vida privada que se tiene derecho a proteger de cualquier intromisión
- ✓ **Rendimiento:** Tasa de producción o número de operaciones realizadas por unidad de tiempo (Throughput) o la velocidad (Tiempo de respuesta/Speed) a la que se puede procesar algo. Se mide en cada caso como “Unidades/peticiones por unidad de tiempo” generalmente en segundos o como tiempo de respuesta respectivamente, en sg, milisegundos, etc.
- ✓ **Resiliencia:** Es la capacidad de absorber o evitar daños sin sufrir un fallo completo. Capacidad de proporcionar y mantener un nivel aceptable de servicio ante la ocurrencia de fallos y desafíos a la operación normal. Nos referimos a cyber-resiliencia ante ataques cibernéticos.
- ✓ **Robustez:** Capacidad de funcionar en presencia de fallos, tensiones, entradas inválidas ...
- ✓ **Seguridad:** Grado de protección de los datos, software y plataforma de tecnología de posibles pérdidas, actividades no permitidas o uso para propósitos no establecidos previamente. También relativa a la seguridad de uso. (Seguridad industrial)
- ✓ **Testabilidad:** facilidad de pruebas y reportes de errores
- ✓ **Usabilidad:** Definido como la facilidad de uso y aprendizaje de un Sistema, Software o Servicio de Tecnología de Información. esfuerzo requerido para aprender, usar, aportar e interpretar Resultados de un programa

1.2 Clasificación de requisitos no Funcionales

Como hemos comentado en la introducción, los requisitos “No Funcionales”, no se refieren directamente a las funciones específicas que entrega el sistema, sino a las propiedades emergentes de éste como la fiabilidad, la respuesta en el tiempo y la capacidad de almacenamiento. De forma alternativa, definen las restricciones del sistema como la capacidad de los dispositivos de entrada/salida y la representación de datos que se utiliza en la interface del sistema.

Los requisitos no funcionales surgen de la necesidad del usuario, debido a las restricciones en el presupuesto, a las políticas de la organización, a la necesidad de interoperabilidad con otros sistemas de software o hardware o a factores externos como los reglamentos de seguridad, las políticas de privacidad, entre otros.

1.2.1 Importancia de clasificar los requisitos no funcionales

El especificar requisitos no funcionales de forma incompleta o inexacta puede ocasionar el fracaso de un proyecto de ingeniería de software.

De hecho no gestionar los requisitos no funcionales es uno de los errores clásicos en la gestión de desarrollo de software definida por Steve McConnell.

Una arquitectura de un sistema, aplicación o proyecto debe también tener en cuenta los requisitos que no aparecen explícitamente en los casos de uso del sistema. Los arquitectos del software, por tanto, tienen la necesidad de entender los requisitos funcionales, para crear una plataforma que apoye dichos requisitos y satisfaga también los requisitos no funcionales.

Existen distintas posibilidades para clasificar y organizar dicho conjunto de requisitos. Una de las más utilizadas es la de Ian Sommerville. Estos diferentes tipos de requisitos se clasifican de acuerdo con sus implicaciones en Producto, Organizacionales y externos.

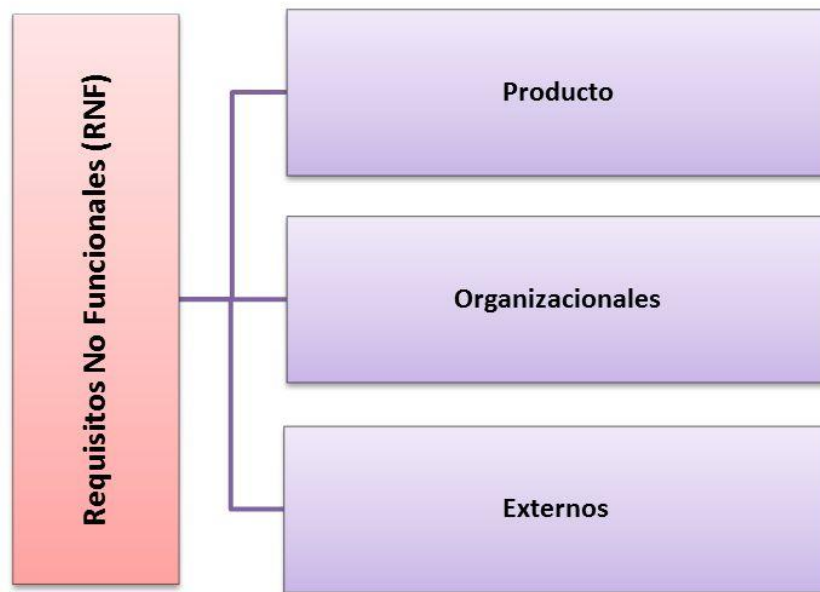


Figura 1.- Clasificación en Niveles de RNF de Ian Sommerville. Ed.9

- ✓ **Requisitos del producto.** Especifican el comportamiento del producto, en relación a:
 - Rendimiento, entendido como la rapidez de ejecución del sistema y la memoria que necesita.
 - Fiabilidad, definida como la tasa de fallos máxima para que el sistema sea aceptable.
 - Portabilidad y usabilidad.
- ✓ **Requisitos organizacionales.** Se derivan de las políticas y procedimientos existentes en la organización del cliente y en la del desarrollador, como por ejemplo:
 - los estándares que deben utilizarse en los procesos.
 - los relativos a la codificación como los lenguajes de programación.
 - la metodología de diseño a utilizar y los requisitos de entrega que especifican cuándo se entregará el producto y su documentación.
- ✓ **Requisitos externos.** Se derivan de factores externos al sistema y de su proceso de desarrollo. Incluyen los requisitos:
 - De interoperabilidad: definen la manera en que el sistema interactúa con los otros sistemas de la organización.
 - Legales: que deben seguirse para asegurar que el sistema opere dentro de la ley.
 - Éticos: Impuestos al sistema para asegurar que será aceptado por el usuario.

En la práctica, la especificación cuantitativa de requisitos es difícil. A los clientes no les es posible (o no tan fácil como en el caso de los Req. Funcionales) traducir sus metas en requisitos cuantitativos. Para algunas de éstas, como las de mantenimiento, no existen métricas que se puedan utilizar; el costo de verificar de forma objetiva los requisitos no funcionales cuantitativos es muy alto.

La siguiente figura recoge un detalle más profundo de la clasificación anterior de RNF de Ian Sommerville.

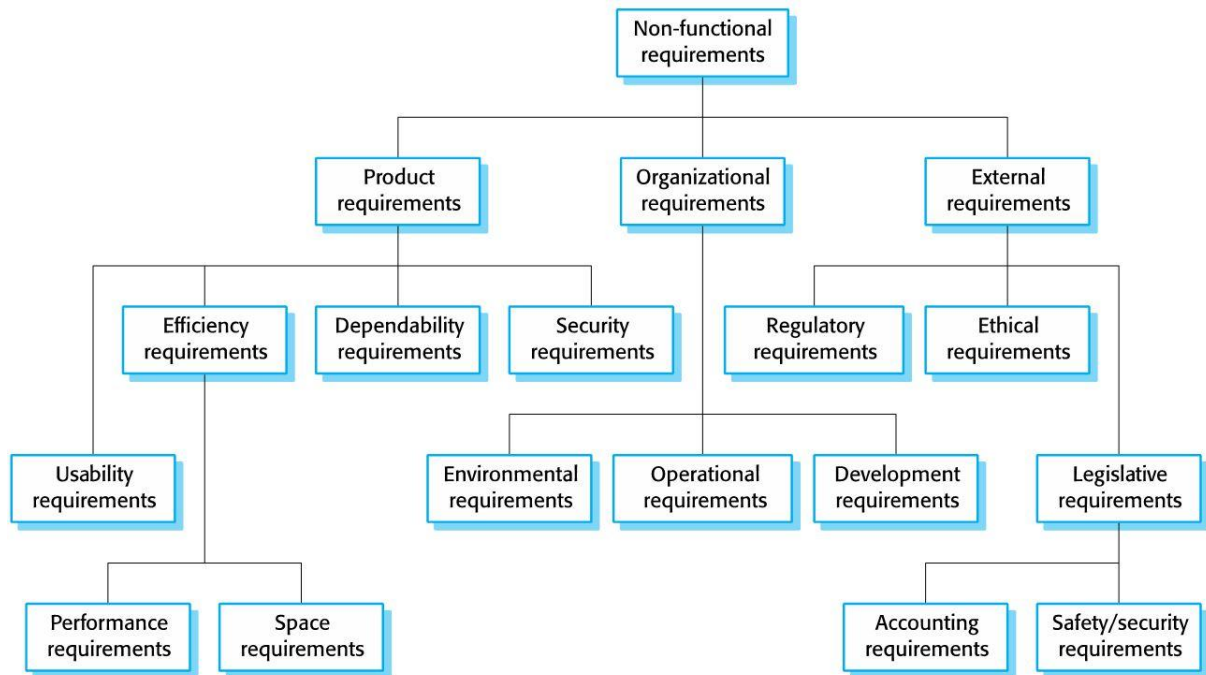


Figura 2.- Clasificación en Niveles de RNF de Ian Sommerville. Ed.9

En principio, los requisitos funcionales y no funcionales se deben diferenciar en el documento de requisitos. En la práctica, esto es difícil. Si un requisito no funcional se declara de forma separada a los funcionales, algunas veces es difícil ver la relación entre ellos. Si se declaran con los requisitos funcionales, es difícil separar las condiciones funcionales y no funcionales e identificar los requisitos que se refieren al sistema como un todo. Se debe hallar un balance apropiado que dependa del tipo de sistema a especificar. Sin embargo, los requisitos que claramente se refieren a las propiedades emergentes del sistema se deben resaltar. Esto se hace colocándolos en una sección aparte o diferenciándolos, de alguna forma, de los otros requisitos del sistema.

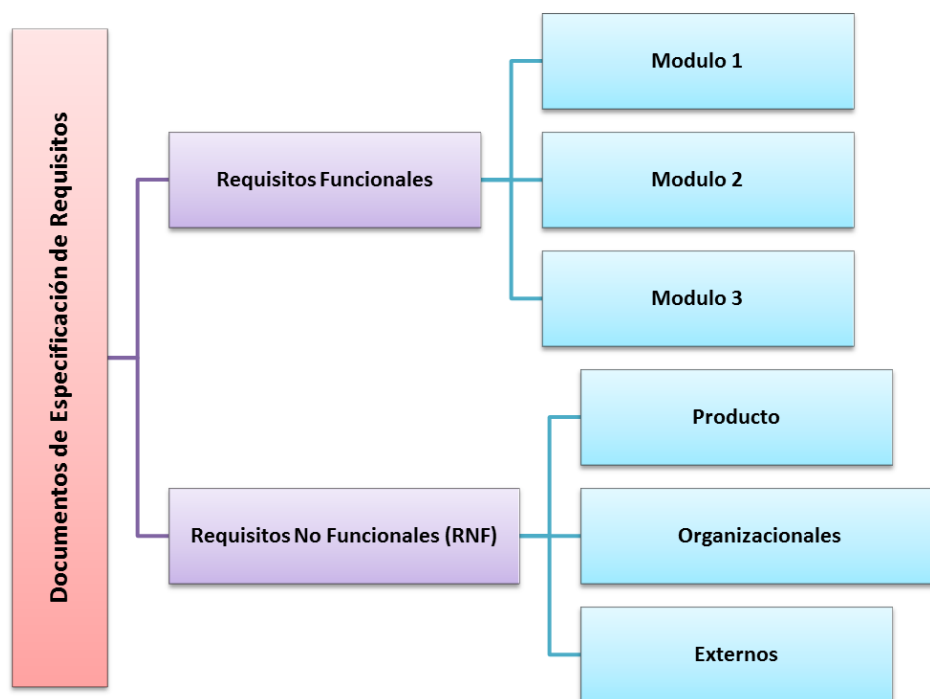


Figura 3.- Esquema de Documento de Especificación de Requisitos

1.2.2 Requisitos no funcionales de producto

Suelen referirse a límites o restricciones sobre el comportamiento del sistema, por lo cual establece límites y restricciones sobre lo que los diseñadores (arquitectos de software) e ingenieros de software pueden hacer.

Algunos de estos requisitos pueden ser fáciles de cuantificar, por ejemplo el desempeño y la confiabilidad, pero otros son más difíciles como por ejemplo usabilidad y adaptabilidad.

Los requisitos de producto pueden clasificarse en (Sommerville):

Usabilidad: La usabilidad se define como el esfuerzo que necesita hacer un usuario para aprender, usar, ingresar datos e interpretar los resultados obtenidos de un software de aplicación. En tiempos recientes, la usabilidad ha adquirido mucha importancia, en especial ante la demanda de desarrollo de software para móviles y tabletas.

Eficiencia: Relacionado con rendimiento (Performance) en cuanto a tiempo de respuesta, número de operaciones por segundo, entre otras medidas, así como al consumo de recursos de memoria, procesador, espacio en disco o red. (Espacio)

Dependibilidad: Engloba varios atributos

- ✓ **Disponibilidad:** Disposición del sistema para prestar servicio correctamente.
- ✓ **Confiabilidad:** Continuidad del servicio prestado por el sistema.
- ✓ **Seguridad industrial:** Ausencia de consecuencias catastróficas para el usuario o el ambiente.
- ✓ **Integridad:** Ausencia de alteraciones inadecuadas al sistema.
- ✓ **Mantenibilidad:** Posibilidad de realizar modificaciones o reparaciones a un proceso sin afectar la continuidad del servicio.

Portabilidad: Capacidad de adaptación a nuevos entornos operativos, cambios de componentes sw, adaptación de la arquitectura por cambios de entorno.

Seguridad: Capacidades funcionales o no funcionales que debe tener un sistema para cumplir atributos en el área de seguridad de tecnología de información, seguridad de datos, seguridad lógica, control de acceso a información (restricciones de acceso), autenticidad de la información y privacidad, entre otros aspectos.

1.2.3 Requisitos no funcionales organizacionales

Se derivan de las políticas y procedimientos de la organización como por ejemplo estándares de procesos o requisitos de implementación.

Pueden incluir metodologías de desarrollo de software, estándares de programación (Buenas prácticas de codificación) y herramientas de soporte al desarrollo de software (por ej. Herramientas CASE) que deben usarse (siguiendo las políticas de la organización), también informes a la dirección que deben proveerse, entre otros.

Las herramientas para la gestión de desarrollo de software que conocemos, se definen como requisitos no funcionales organizacionales.

Los requisitos organizacionales se clasifican según Sommerville:

- ✓ **Requisitos de Entorno (Environment):** Hace referencia específicamente a los procedimientos de mantenimiento y administración del entorno de desarrollo de software. Esta administración también incluye los procedimientos para gestionar los entornos de pruebas, integración Continua, etc
 - Entorno de desarrollo de software (ambiente de desarrollo)
 - Entorno de pruebas de software (ambiente de pruebas)
- ✓ Describen los estándares tecnológicos que se utilizarán por el sistema y que son requeridos por el negocio. En el caso de interoperar con sistemas externos estará relacionado con los requisitos NF externos de Interoperabilidad.
- ✓ **Requisitos operacionales (Delivery):** Procedimientos operativos que describen como será usado el sistema dentro del contexto de la organización.
- ✓ **Requisitos de desarrollo (Development):** Engloba aspectos relativos al desarrollo e implementación del sistema como
 - Lenguaje de programación a usar
 - Estándares de codificación
 - Patrones (y antipatrones) de diseño y programación
 - Herramientas para gestionar el desarrollo de software
 - Etc...

1.2.4 Requisitos no funcionales externos

Estos derivan del entorno organizacional (entorno no técnico) en el cual se desarrolla el sistema y pueden hacerse tanto sobre el producto (el software desarrollado) o también sobre el proceso de desarrollo de software.

Este tipo de requisitos incluyen limitaciones de índole económica, como por ejemplo el presupuesto del proyecto de software, interacción o necesidad del sistema de inter-operar con otros sistemas, requisitos regulatorios en el área de salud, seguridad industrial o protección de datos, requisitos legales concernientes con licencias, regulaciones o certificaciones que necesita el producto según la industria en el que se desempeñe, entre otros.

Sommerville a su vez clasifica estos requisitos en:

- ✓ **Requisitos Regulatorios:** Las restricciones, licencias y leyes o normas aplicables a un producto o negocio específico, por ejemplo para operar en banca, eléctricas y en general cualquier negocio regulado. Desde el punto de vista normativo serían aquellas leyes específicas del negocio, por ejemplo en el área de contabilidad (normas contables y estándares financieros), requisitos de seguridad industrial (para sistemas críticos).
- ✓ **Requisitos éticos:** Requisitos que aseguran que el sistema será aceptable para el usuario, público en general y se adapta a las costumbres de la sociedad en la que se desenvuelve o a la que presta servicios.
- ✓ **Requisitos legislativos:** Características que debe cumplir el sistema para cumplir con Leyes y reglamentos que establecen que debe hacer el sistema y como debe hacerlo para cumplirlas. El foco de un sistema o nueva funcionalidad puede ser exclusivamente para cumplir una normativa por ejemplo la Ley de Protección de datos, el nivel de Accesibilidad, web, etc.

1.2.5 Importancia de clasificar los requisitos no funcionales

El especificar requisitos no funcionales de forma incompleta o inexacta puede ocasionar el fracaso de un proyecto de ingeniería de software.

De hecho, no gestionar los requisitos no funcionales es uno de los errores clásicos en la gestión de desarrollo de software definida por Steve McConnell.

Una arquitectura de un sistema, aplicación o proyecto debe también tener en cuenta los requisitos que no aparecen explícitamente en los casos de uso del sistema. Los arquitectos del software, por tanto, tienen la necesidad de **entender los requisitos funcionales**, para crear una plataforma que apoye dichos requisitos y satisfaga también los requisitos no funcionales.

Hay otras clasificaciones más sencillas que, por ejemplo, identifican en tres áreas los requisitos **no funcionales**:

- ✓ Técnicos
- ✓ Negocio
- ✓ Calidad

Un ejemplo de esta aproximación se muestra a continuación.

1. **Técnicos:** Estos son familiares para todos. Se limitan las opciones de diseño mediante la especificación de algunas tecnologías que se deben utilizar. "Sólo tenemos los desarrolladores de Java, por lo que debemos desarrollar en Java." "La base de datos existente se ejecuta en Oracle." Estos requisitos son por lo general, "no negociables".
2. **De negocio:** Para los negocios, no hay razones técnicas. Por ejemplo, "A fin de ampliar nuestra base de clientes potenciales, se debe interactuar con las herramientas de de "gestión Documental XXX"." Otro ejemplo es "El proveedor de nuestro middleware ha aumentado sus precios a niveles prohibitivos, por lo que nos estamos moviendo a una versión de código abierto." La mayoría de estos requisitos también son "no negociables".
3. **De calidad:** definir los requisitos de una aplicación en términos de escalabilidad, disponibilidad, facilidad de cambio, la portabilidad, facilidad de uso, por rendimiento, etc., que como se ha comentado anteriormente en inglés se suele referir a ellos como "-ilities". Por ejemplo procesar 1000 sesiones concurrentes.

Una clasificación un poco más elaborada es la denominada FURPS.

1.2.6 FURPS

FURPS es un acrónimo que representa un modelo para clasificar atributos de calidad de software (requisitos funcionales y no funcionales):

- ✓ **Functionality:** Funcionalidad - Capacidad (tamaño y generalidad del conjunto de funcionalidades), Reutilización (compatibilidad, interoperabilidad, portabilidad), seguridad
- ✓ **Usability (UX):** Usabilidad (UX) - Factores Humanos, Estética, Coherencia, Documentación, Responsividad
- ✓ **Reliability:** Fiabilidad - Disponibilidad (Frecuencia de fallos (Robustez / Durabilidad / Resistencia), Duración e impacto de los fallos, (Recuperabilidad / Capacidad de Supervivencia)), Predictibilidad (Estabilidad), Precisión (Frecuencia / Gravedad del Error)
- ✓ **Performance:** Rendimiento - Velocidad, eficiencia, consumo de recursos (alimentación, memoria RAM, caché, etc.), Throughput, capacidad y escalabilidad
- ✓ **Supportability:** Facilidad de soporte, facilidad de mantenimiento, sostenibilidad, velocidad de reparación - Testabilidad, flexibilidad (modificabilidad, configurabilidad, adaptabilidad, extensibilidad y modularidad), facilidad de instalación y localizabilidad

El modelo, desarrollado en Hewlett-Packard, fue inicialmente elaborado públicamente por Grady y Caswell. "**FURPS+**" ahora es ampliamente utilizado en la industria del software. El "+" se añadió posteriormente al modelo después de varias campañas en HP para extender el acrónimo para enfatizar e identificar categorías adicionales que generalmente representan restricciones relativos a:

- ✓ Diseño
- ✓ Implementación
- ✓ Interfaz
- ✓ Requisitos físicos

Ejemplos serían:

- ✓ Un requisito de diseño, a menudo denominado restricción de diseño, especifica o restringe las opciones para diseñar un sistema. Por ejemplo, si especifica que se requiere una base de datos relacional, se trata de una restricción de diseño.
- ✓ Un requisito de implementación especifica o restringe la codificación o construcción de un sistema. Los ejemplos pueden incluir estándares requeridos, lenguajes de implementación y límites de recursos.
- ✓ Un requisito de interfaz especifica un elemento externo con el que un sistema debe interactuar, o restricciones sobre formatos u otros factores utilizados dentro de dicha interacción.
- ✓ Un requisito físico especifica una restricción física impuesta al hardware utilizado para alojar el sistema, por ejemplo: la forma, el tamaño o el peso del sistema.

1.3 Rendimiento - Performance

En sistemas de información, el rendimiento (performance) es la cantidad de trabajo realizado por un sistema informático. Dependiendo del contexto, un alto rendimiento puede involucrar uno o más de los siguientes:

- ✓ Tiempo de respuesta corto para un determinado trabajo
- ✓ Alto Throughput (tasa de procesamiento alta)
- ✓ Baja utilización de los recursos informáticos
- ✓ Alta disponibilidad del sistema informático o de la aplicación
- ✓ Compresión y descompresión de datos rápido o muy compactado
- ✓ Alto ancho de banda
- ✓ Tiempo corto para una transmisión de datos

El rendimiento de cualquier sistema informático puede evaluarse en términos técnicos medibles, utilizando una o más de las métricas enumeradas anteriormente. De esta manera el rendimiento puede ser comparado con otros sistemas o con el mismo sistema antes y después de los cambios.

Es decir, “cómo de bien está haciendo su trabajo”.

El rendimiento de sistemas, particularmente el tiempo de respuesta de la aplicación software, es un aspecto de la calidad del software importante.

El tiempo de respuesta es el tiempo total que se tarda en responder a una solicitud de servicio, pudiendo ser desde el acceso a disco hasta la carga de una página web compleja. El tiempo de respuesta es la suma de tres factores:

- ✓ Tiempo de servicio - Cuánto tiempo se tarda en hacer el trabajo solicitado.
- ✓ Tiempo de espera - El tiempo que la solicitud tiene que esperar por las solicitudes en cola antes de que llegue a ejecutarse.
- ✓ Tiempo de transmisión - Cuánto tiempo se tarda en mover la solicitud y la respuesta extremo a extremo.

1.3.1 La optimización del rendimiento (tunning)

El ajuste del rendimiento es la mejora del rendimiento del sistema. Esto es típicamente una aplicación informática, pero los mismos métodos se pueden aplicar a los mercados económicos, burocracias u otros sistemas complejos. La motivación para tal actividad será un problema de rendimiento, que puede ser real o anticipado. La mayoría de los sistemas responderán a una mayor carga con cierto grado de rendimiento decreciente. La capacidad de un sistema para aceptar una carga más alta sin degradarse se llama escalabilidad, y modificar un sistema para manejar una carga más alta es sinónimo de ajuste de rendimiento (performance Tuning)

La mejora sistemática de rendimiento sigue estos pasos:

- ✓ Evaluar el problema y establecer valores numéricos que categoricen el comportamiento aceptable.
- ✓ Medir el rendimiento del sistema antes de la modificación.
- ✓ Identificar la parte del sistema que es crítica para mejorar el rendimiento. Esto se denomina el cuello de botella.
- ✓ Modificar esa parte del sistema para eliminar el cuello de botella.
- ✓ Medir el rendimiento del sistema después de la modificación.
- ✓ Si la modificación hace que el rendimiento sea mejor, adoptarlo. Si la modificación empeora el rendimiento, se vuelve a poner como estaba.

1.3.2 Rendimiento percibido

El rendimiento percibido, en ingeniería informática, se refiere a la rapidez con que una función de software parece realizar su tarea. El concepto se aplica principalmente a los aspectos de aceptación de los usuarios.

La cantidad de tiempo que tarda una aplicación en arrancar, o un archivo para descargar, no se hace más rápido mostrando una pantalla de inicio (consulte Pantalla de bienvenida) o un cuadro de diálogo de progreso de archivo. Sin embargo, satisface algunas necesidades humanas: parece más rápido para el usuario, así como proporcionar una señal visual para hacerles saber que el sistema está manejando su solicitud.

En la mayoría de los casos, el aumento del rendimiento real aumenta el rendimiento percibido, pero cuando el rendimiento real no puede aumentarse debido a limitaciones físicas, se pueden utilizar diversas técnicas para aumentar el rendimiento percibido al costo de disminuir ligeramente el rendimiento real.

1.4 Seguridad.

Cualquier organización que expone sus servicios informáticos debe realizar un esfuerzo significativo para asegurar que la información y recursos están protegidos. Internet es un factor primordial en la comunicación y también un evidente riesgo potencial de acceso y mal uso de los servicios e información disponibles. Obviamente, se catalogan sistemas más críticos que otros donde su seguridad debe de ser muy significativa, pero en general todas las aplicaciones, especialmente las aplicaciones Web, deben de estar protegidas y aseguradas ante los principales ataques.

El tema de seguridad se trata más en detalle en los temas 120 ***La seguridad en el nivel de aplicación. Tipos de ataques y protección de servicios web, bases de datos e interfaces de usuario*** y 45 y 46 relativos a ***Seguridad de sistemas***, entre otros.

En una aplicación dividimos la seguridad en:

- **Disponibilidad:** Propiedad o característica de los activos consistente en que las entidades o procesos autorizados tienen acceso a los mismos cuando lo requieren.
- **Autenticidad:** Propiedad o característica consistente en que una entidad es quien dice ser o bien que garantiza la fuente de la que proceden los datos
- **Integridad:** Propiedad o característica consistente en que el activo de información no ha sido alterado de manera no autorizada.
- **Confidencialidad:** Propiedad o característica consistente en que la información ni se pone a disposición, ni se revela a individuos, entidades o procesos no autorizados.
- **Trazabilidad:** Propiedad o característica consistente en que las actuaciones de una entidad pueden ser imputadas exclusivamente a dicha entidad.

1.4.1 Medidas de prevención contra las vulnerabilidades

El análisis de las vulnerabilidades potenciales es un objetivo básico para el incremento de la seguridad en las aplicaciones web, que en ocasiones es subestimado como factor de riesgo crítico. El mantener parámetros que no son verificados, roles sin controlar, desbordamientos que se producen

en la memoria son algunas de las situaciones que pueden provocar brechas de seguridad en las aplicaciones.

Los desarrollos que se realizan comercialmente presentan las mismas deficiencias. De ahí, que necesiten una actualización constante para asegurar la reparación de bugs que se van encontrando en el tiempo de vida de las aplicaciones. Con frecuencia, se asemeja la seguridad de una aplicación con la seguridad de la plataforma donde se ejecuta. El esfuerzo que se realice para aumentar la seguridad tanto a nivel de desarrollo como de diseño, debe de ser un esfuerzo a nivel de grupo. Hay que incluir en el desarrollo un compromiso para el aseguramiento de las aplicaciones. Los servidores de producción y los otros sistemas deben mantenerse regularmente con las últimas firmas para garantizar que están libres de vulnerabilidades a nivel de sistema. Se recomienda seguir las siguientes indicaciones destinadas a mantener una aplicación alejada de las vulnerabilidades, como requisitos No Funcionales:

- **Entornos de trabajo diferenciados.** Es muy aconsejable mantener entornos separados del de producción. Los entornos de calidad y de desarrollo son frecuentemente manipulados, por lo que entrarán en conflicto con los datos mostrados por producción. Es primordial que sus entornos estén bien diferenciados, a ser posible mediante cortafuegos/Firewalls. Esta situación es prioritaria cuando el entorno de producción es una zona interna o de transición y accesible desde Internet. Si el entorno de producción tramita y ejecuta procesos, hay que asegurar los mismos tanto a nivel de sistema como de aplicación previamente a incluir nuevos procesos. Este principio hay que mantenerlo cada vez que se actualice o mejore.
- **Distribución de las actualizaciones.** Con cierta frecuencia las aplicaciones realizan actualizaciones para cubrir posibles deficiencias que se detectan a lo largo del ciclo de vida de la aplicación. Dependiendo del carácter de estas aplicaciones, si son externas, las actualizaciones son inmediatas, si son internas se debe además examinar el código de las aplicaciones creadas internamente e introducir las actualizaciones o nuevas versiones a medida que vaya siendo necesario.
- **Contramedidas temporales.** Es necesario realizar un protocolo de respuesta ante posibles situaciones que entrañen riesgo para las aplicaciones. En este protocolo se pueden indicar acciones a realizar en función de la gravedad del riesgo detectado (cerrar puertos, bloquear direcciones, routers). Es muy importante, para lograr eficiencia en la respuesta, saber interpretar los efectos que delimitan el riesgo y mantener a todo el equipo de trabajo involucrado plenamente informado del protocolo de respuesta ante la activación del mismo.
- **Modo de Fallos.** Se debe de disponer de un plan que permita mantener en funcionamiento el proceso crítico si se da la situación que una aplicación falle. Es decir, debe de permitir una transacción que permita enrutarse, de forma ajena a la aplicación, o bien permitir realizar de forma manual las funciones descritas en la aplicación. Lo más importante es mantener el criterio de " cierre ante fallo". Una aplicación no debe abrirse si detecta un fallo. Si lo hace, corre el riesgo de permitir todo tipo de acciones en el sistema. Las aplicaciones o sus comprobaciones internas siempre deben cerrarse al fallar.
- **Conocimiento de desarrolladores y políticas.** Es muy importante que todas las personas involucradas en un desarrollo tengan el conocimiento necesario de las herramientas disponibles y de las políticas necesarias para crear códigos seguros. Las políticas de seguridad deben de ser generales, por lo tanto deben de mantenerse en un nivel que permita aplicarse al global del conjunto de proyectos.

1.4.2 Interfaz de Seguridad en Métrica

En métrica, se trata especialmente la seguridad a través del interfaz de seguridad, que permite la comunicación entre las tareas de desarrollo y las tareas de análisis y gestión de riesgos.

- ✚ La dirección aporta los servicios necesarios y la calidad de la seguridad deseada.
- ✚ El equipo de desarrollo aporta los elementos técnicos que materializan la aplicación.
- ✚ El equipo de análisis de riesgos aporta un juicio crítico sobre la seguridad del sistema.

Es decir que se manejan simultáneamente diferentes requisitos:

- ✚ de servicio, procedentes de la dirección
- ✚ técnicos, procedentes del equipo de desarrollo

de seguridad, procedentes del equipo de análisis de riesgos

Esto da pie a una interrelación continua entre el equipo de desarrollo y el equipo de seguridad que, a través de la interfaz de seguridad, van cerrando cada etapa de Métrica. Es importante destacar que el alcance de un nivel de seguridad puede requerir modificaciones en los componentes técnicos del sistema; al tiempo que los detalles técnicos pueden alterar el análisis de seguridad.

En cualquier caso, el punto de acuerdo entre los componentes (activos) y el estado de seguridad (impacto y riesgo) debe ser aprobado por la dirección de la Organización.

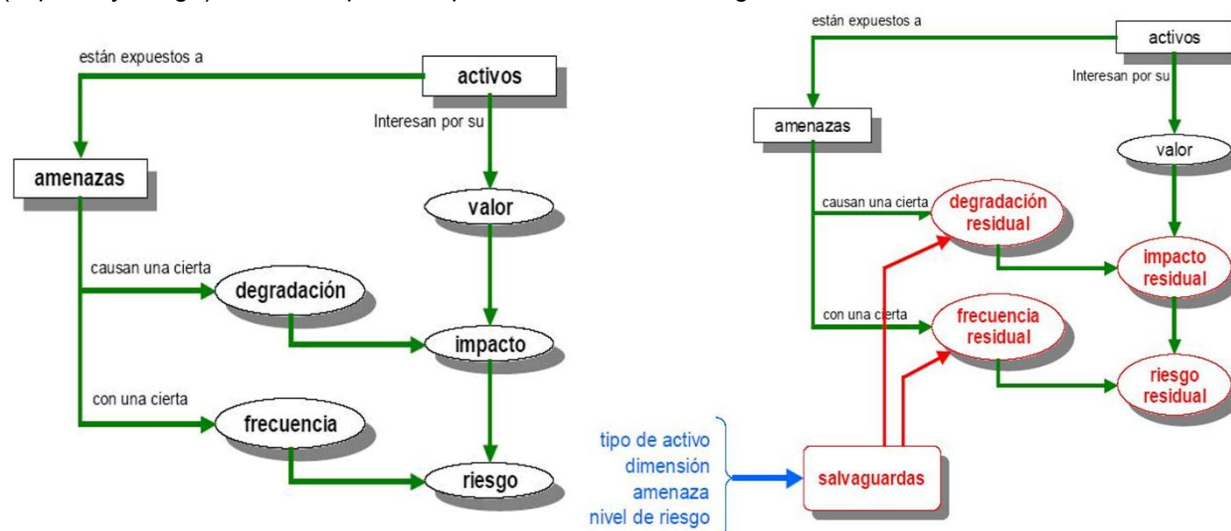


Figura 4 .- Modelo de análisis y Gestión de Riesgos como proceso dinámico aplicando salvaguardas. Fuente: Magerit

En Métrica se distingue entre la seguridad del proceso de desarrollo (tareas SPD) y la seguridad del sistema de información (SSI). Las tareas de la interfaz se organizan según su pertenencia a uno u otro objetivo de seguridad. Estos se deben reflejar y concretar en forma de requisitos no funcionales, salvo que se asuman intrínsecamente como parte de la metodología o del proceso de desarrollo.

El objetivo es incorporar a los Sistemas mecanismos de seguridad adicionales a los que se proponen en los procesos principales. (Requisitos funcionales ASI 2.1), previamente al desarrollo del mismo.

Son los relativos al Análisis y Gestión de Riesgos. Por ejemplo no sobrepasar costes asignados, no disponer de suficientes recursos para realizar las tareas, la funcionalidad no responde a las expectativas, desviaciones con respecto a la planificación, o elaboración de planes de contingencia (respaldo, emergencia y restauración).

Este marco se desarrolla fundamentalmente con Magerit y los criterios de Seguridad, Normalización y Conservación (**SNC**) que relacionan requisitos legales (relativos a la validez y eficacia de los procedimientos administrativos que utilicen los medios electrónicos informáticos y telemáticos, y de protección de los datos de carácter personal, entre otros) con criterios técnicos (organizativos o tecnológicos), fundados en normas de autoridad y amplia aceptación.

La interfaz de Seguridad hace posible incorporar durante la fase de desarrollo las funciones y mecanismos que refuerzan la seguridad del nuevo sistema y del propio proceso de desarrollo, asegurando su consistencia y seguridad.

El análisis de los riesgos constituye una pieza fundamental en el diseño y desarrollo SI seguros.

Los riesgos contemplados en la interfaz de Seguridad de MÉTRICA Versión 3 son de distinta índole:

- ✚ naturales (inundaciones, incendios, etc.) o
- ✚ lógicos (fallos propios, ataques externos, virus, etc.)

De lo anterior se desprende que existen dentro de la interfaz dos tipos de actividades diferenciadas:

- ✚ Actividades relacionadas con la seguridad intrínseca del sistema de información - representadas en la parte inferior de los gráficos (SSI)
- ✚ Actividades que velan por la seguridad del propio proceso de desarrollo del sistema de información -representadas en la parte superior de los gráficos. (SPD)

Si en la organización ya existe un plan de seguridad o una metodología de análisis y gestión de riesgos como por ejemplo MAGERIT, para cada sistema de información deberán analizarse las necesidades de seguridad del sistema respecto al método vigente, y determinar las diferencias si las hubiera, así como aquellas necesidades concretas que no se encuentren recogidas, estableciendo así el plan de seguridad del SI.

Si no existe un plan de seguridad en la organización habrá que desarrollarlo desde el principio. El plan recogerá además las medidas de seguridad activas o preventivas y reactivas, en respuesta a situaciones en que se produce un fallo reduciendo su efecto, relacionadas con la seguridad del sistema de información y del proceso de desarrollo.

Las valoraciones sobre la seguridad deben ser realizadas en función de las características del sistema: complejidad, tamaño, incertidumbre, participantes, etc. por los responsables de la seguridad del sistema de información, quienes se apoyarán para sus decisiones en su conocimiento y experiencia en la materia sin perder de vista además que, al ser finitos los recursos, no pueden asegurarse todos los aspectos del desarrollo de los sistemas de información, por lo que habrá que aceptar un determinado nivel de riesgo concentrándose en los aspectos más comprometidos o amenazados, que serán diferentes según las circunstancias.

1.5 Privacidad

La privacidad puede ser definida como el ámbito de la vida privada que se tiene derecho a proteger de cualquier intromisión, que se desarrolla en un espacio reservado, el cual debe mantenerse confidencial. También se aplica a la cualidad de privado.

En el marco de las aplicaciones, la privacidad se refiere a la información personal de los clientes, usuarios y personas relacionadas con el mismo y cuya información debe protegerse.

Este requisito está muy relacionado con la seguridad de la información y más en concreto con la confidencialidad, entendiéndolo por confidencialidad la propiedad de la información, por la que se garantiza que está accesible únicamente a personal autorizado a acceder a dicha información. La confidencialidad ha sido definida por la Organización Internacional de Estandarización (ISO) en la norma ISO/IEC 27002 como "garantizar que la información es accesible sólo para aquellos autorizados a tener acceso" y es una de las piedras angulares de la seguridad de la información.

La Unión Europea (UE) promulgó el Reglamento General de Protección de Datos (RGPD), el cual se encuadra en la reforma normativa realizada con el objetivo de garantizar unos estándares de protección de datos elevados y adaptados a la realidad digital del mundo actual. Este Reglamento, que deroga la Directiva 95/46/CE, fue aprobado por el Parlamento Europeo en abril de 2016 y entró en vigor 20 días después de su publicación en el Diario Oficial de la Unión Europea, el 4 de mayo de 2016. Su ámbito de aplicación se extiende a todos los países miembros de la Unión Europea y se aplicará directamente en todos ellos a partir del 25 de mayo de 2018.

En el tema 35 del temario relativo a la LOPD se amplía el tema de la protección de datos.

1.5.1 Paradoja de la privacidad

La paradoja de la privacidad es un fenómeno en el que los usuarios en línea afirman que están preocupados por su privacidad pero se comportan como si no lo estuvieran.

En comparación con los adultos, los jóvenes tienden a revelar más información sobre las redes sociales. Sin embargo, esto no significa que no están preocupados por su privacidad. En una entrevista en televisión sobre Facebook, Una estudiante señaló sus preocupaciones sobre la divulgación de información personal en línea.

Sin embargo, cuando la periodista pidió ver su página de Facebook, había publicado la dirección de casa, números de teléfono, e imágenes de su hijo menor en la página. Susan B. Barnes denominó este fenómeno como "**paradoja de la privacidad**" para referirse a la frontera ambigua entre el espacio privado y el público en las redes sociales.

Cuando se prestan servicios en los que se requiere información del usuario, debe recogerse en un documento, denominado Política de privacidad, las condiciones y uso legítimo que se va a hacer con dicha información, conteniendo al menos los datos que se van a recoger y gestionar, el motivo y uso de los mismos, así como de las medidas de seguridad que se van a implementar para proteger dichos datos.

También deberán definirse una serie de funcionalidades o mecanismos para poder gestionar el usuario dichos datos para proteger su privacidad, pudiendo incluso cancelarlos aunque suponga la finalización del servicio o relación con el prestador del mismo.

2 Calidad de requisitos no Funcionales: Medidas

Todos los Servicios de Tecnología de Información (TI) en algún punto de su ciclo de vida, necesitan considerar los Requisitos no funcionales y las pruebas asociadas a los mismos.

Para algunos proyectos, estos requisitos implican una cantidad considerable de trabajo y esfuerzos, mientras que para otros no.

Con frecuencia, los Requisitos no funcionales son ignorados o subestimados en la fase de análisis de requisitos. El error, termina identificándose en la fase de implementación, durante las pruebas de sistema o una vez en producción, cuando remediarlos implica más trabajo y costo, pudiendo ocasionar que los sistemas o aplicaciones no sean adoptados por los usuarios y clientes. En el siguiente gráfico se representa el coste relativo de corregir un error. Si el coste relativo de corregir un error en la fase de análisis, el coste de corregirlo en el resto de fases se multiplica por un factor de 3 a 6 en el Diseño, un factor de 10 en la Construcción, un factor de 14 a 40 en las Pruebas de Desarrollo, un factor de 30 a 70 en las pruebas del sistema y entre 40 y 1000 cuando el sistema está en explotación.

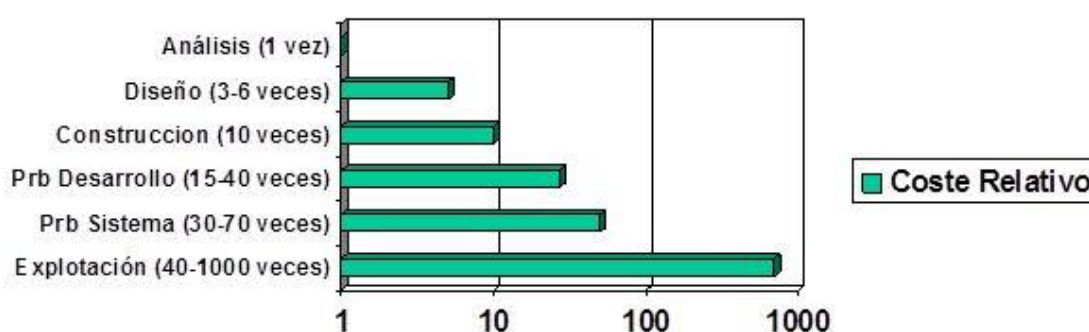


Figura 5.- coste relativo de corregir un error según la fase de desarrollo Software en la que se detecta

La mayoría de las definiciones relativas a la calidad de un sistema exigen el cumplimiento de los requisitos como la de Roger S. Presmman:

"Conformidad explícita con los requisitos funcionales y de rendimiento, normas de desarrollo explícitamente documentadas, y las características implícitas que se esperan de todo Software desarrollado profesionalmente".

Eso implica la necesidad de poder cuantificar explícitamente los requisitos y verificar que cualquier solución es compatible con ellos

Un fenómeno interesante que suele ocurrir es que normalmente se alcanzan los objetivos mensurables, por lo tanto, a menos que fijen valores poco realistas, los requisitos se suelen cumplir.

Para ello es importante saber qué medidas existen, y que los valores elegidos pueden tener un impacto muy significativo sobre la cantidad de trabajo durante el desarrollo, así como el número de alternativas y diseños arquitectónicos por los que el equipo de desarrollo puede optar para cumplir los requisitos

2.1 Métricas, Medidas y Calidad de requisitos no Funcionales

Los requisitos no funcionales deben ser medibles, evitando la caracterización subjetiva: buena, óptima, mejor, etc.

Los valores no deben especificarse aleatoriamente

- ✓ Debe tener racionalidad, es decir una justificación técnica, de negocio, etc.
- ✓ Las partes interesadas deben entender los compromisos
- ✓ Importante para clasificar y priorizar los requisitos

Es poco probable que los números precisos se conozcan al principio del proceso de toma y análisis de requisitos, por lo tanto, si bien no es recomendable ralentizar o retrasar el proceso de toma de requisitos inicial, hay que asegurarse de que se identifican los atributos de calidad y negociar valores precisos más adelante durante el proceso de análisis, diseño o implementación.

Tradicionalmente se utiliza el término “medidas” de manera genérica, pero en realidad existe una distinción entre medidas y métricas.

En ingeniería del software una métrica es cualquier medida o conjunto de medidas que permiten caracterizar un software o sistema de información

Una medida es la cantidad que resulta de medir una magnitud. Para definir una medida, hay que definir lo que se mide (es decir, la “calidad”), las unidades métricas utilizadas con los valores de medición, y “CÓMO” se mide.

Un ejemplo de medida sería el “número de fallos en un período de tiempo concreto”, 1sg a las 12:00 del día 1 de enero, es decir, ¡una observación!.

La métrica sería la generalización de esta medida, número de fallos por segundo, y su comparación respecto al umbral definido un criterio de calidad.

Por ejemplo, considerando la fiabilidad, una métrica sería el tiempo medio entre fallos. A continuación se enumeran, sin ánimo de ser completamente exhaustivos una serie de métricas y medidas respecto a los requisitos no funcionales más importantes y genéricos.

2.1.1 Componentes de un modelo de calidad

Un modelo de calidad consiste en una jerarquía de componentes compuesta por:

- ✓ **Grupos de factores de calidad:** que son grupos que se asocian por factores de calidad relacionados.
- ✓ **Factores de calidad:** son aspectos de alto nivel, atributos o características de una aplicación, componente, centro u organización.
- ✓ **Subfactores de calidad:** son factores de calidad de nivel inferior que son componentes de otros factores o subfactores de calidad.
- ✓ **Criterios de Calidad:** son descripciones específicas que proporcionan evidencia para o contra la existencia de un factor o un subfactor de calidad específico.
- ✓ **Medidas de Calidad:** son métricas que cuantifican los criterios de calidad y los convierte por tanto en mensurables, objetivos e inequívocos.

Estos componentes así como su relación se ilustran en la siguiente figura:

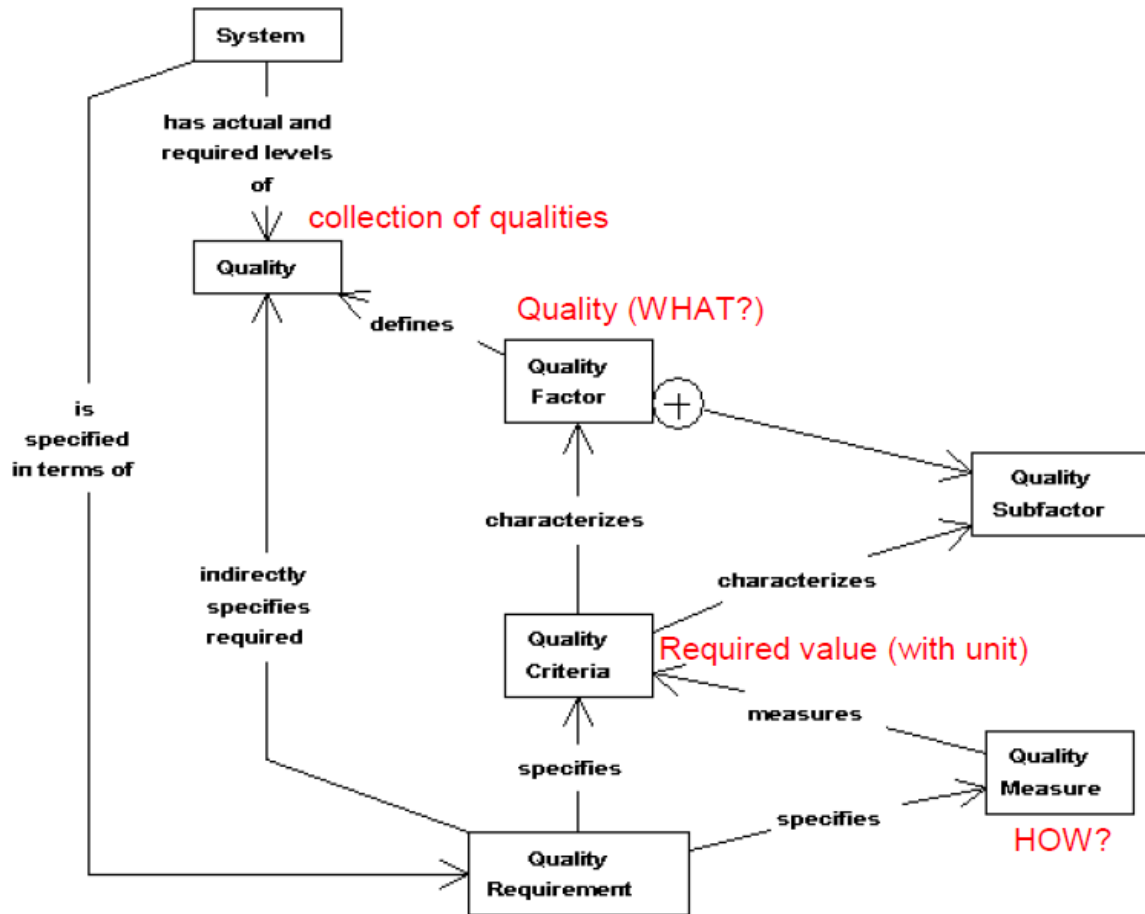


Figura 6.- Relaciones de los Componentes de un Modelo de Calidad

Las distintas medidas de calidad asociadas a requisitos no funcionales deben ser coherentes con el modelo de calidad.

2.1.2 Medidas de rendimiento - Performance

Relativas al rendimiento hay muchas medidas que se pueden emplear dependiendo de las características específicas o RNF que se quieran medir y monitorizar, por ejemplo:

- ✓ Tiempo de respuesta
- ✓ Número de eventos procesados / negados en algún intervalo de tiempo,
- ✓ Throughput
- ✓ Capacidad de almacenamiento
- ✓ Tasa de uso
- ✓ Fluctuación (jitter),
- ✓ Pérdida de información
- ✓ Latencia...

Usualmente, estas medidas deben ir complementadas con probabilidades, intervalo de confianza, umbrales etc.

Estas medidas pueden ser modeladas y/o simuladas, principalmente en el nivel de arquitectura y ser utilizadas como una predicción de rendimiento. Para ello se pueden usar los siguientes mecanismos:

- ✓ Modelo de colas (LQN), álgebra de procesos, redes estocásticas de Petri
- ✓ Tasas de llegada, proceso, salida, distribución de solicitudes de servicio, etc.
- ✓ Análisis de sensibilidad, análisis de escalabilidad, etc..

Una herramienta que permite medir estos RNF y configurar pruebas para ello es Jmeter.

Estos requisitos pueden variar de un sistema a otro:

- ✓ Sistema Web: Número de peticiones concurrentes por segundo
- ✓ Video juegos: Número de imágenes 3D por segundo

A continuación se relacionan unos cuantos ejemplos de requisitos de rendimiento.

- ✓ El sistema deberá poder procesar 100 transacciones de pago por Segundo en la carga máxima.
- ✓ En la carga de trabajo estándar, el uso de la CPU será inferior al 50%, permitiendo un 50% de carga para trabajos en background.
- ✓ La producción de un informe simple llevará menos de 20 segundos para el 95% de los casos.
- ✓ Desplazar una página hacia arriba o hacia abajo en un documento de 200 páginas como máximo será 1 segundo.
- ✓ El tiempo de respuesta máximo del servicio de consulta de datos de identidad síncrono no excederá de los dos segundos, siendo en media inferior a un segundo.
- ✓ Procesar una petición asíncrona en la PID no debe exceder de los 10 sg, en el caso de contar con 1000 solicitudes en cada petición.

2.1.3 Medidas de Fiabilidad

El objetivo de las medidas relativas a la fiabilidad de un sistema es medir el grado con el que el sistema se comporta según lo requerido o especificado. Estas medidas deben cubrir aspectos como:

- ✓ Resistencia al fallo
- ✓ Capacidad para realizar una función requerida en las condiciones establecidas durante un período de tiempo especificado (Bajo presión o alta carga)

La fiabilidad es una cualidad muy importante para sistemas críticos, continuos o científicos.

Se puede medir usando las siguientes medidas:

- ✓ Probabilidad de que el sistema realice su función requerida para un intervalo bajo condiciones establecidas
- ✓ **MTBF**: Tiempo medio transcurrido entre los fallos
- ✓ Tasa de defectos
- ✓ Grado de precisión para los cálculos

A continuación se relacionan unos cuantos ejemplos de requisitos de fiabilidad:

- ✓ La precisión de los cálculos será de al menos 1/106.
- ✓ La tasa de defectos del sistema será inferior a 1 fallo por cada 1000 horas de operación.
- ✓ No más de 1 por 1.000.000 transacciones resultará en un fallo requiriendo un reinicio del sistema.
- ✓ La tasa de respuestas fallidas (SOAP-FAULT) será inferior a un 1% de las tramitadas debidas a fallos del sistema.

2.1.4 Medidas de Disponibilidad

Las medidas de disponibilidad van a hacer referencia al porcentaje de tiempo que el sistema está funcionando correctamente

Se puede calcular basándose en el tiempo medio hasta el fallo (MTTF) y el tiempo medio de reparación (MTTR)

- ✓ **MTTF**: Mean time to failure. Tiempo medio hasta el fallo.
- ✓ **MTTR**: (Mean time to recovery) - tiempo necesario para reanudar la operación después de un fallo. También se le llama Mean time to repair, más asociado a temas HW.

- ✓ **MTBF:** (Mean time between failures) - Tiempo medio transcurrido entre los fallos. Se mide como el tiempo total en el que el sistema está disponible dividido entre el número de fallos. También como la suma de "MTTF" + "MTTR"
- ✓ **Disponibilidad** = $MTTF / (MTTF + MTTR) = MTTF / MTBF$

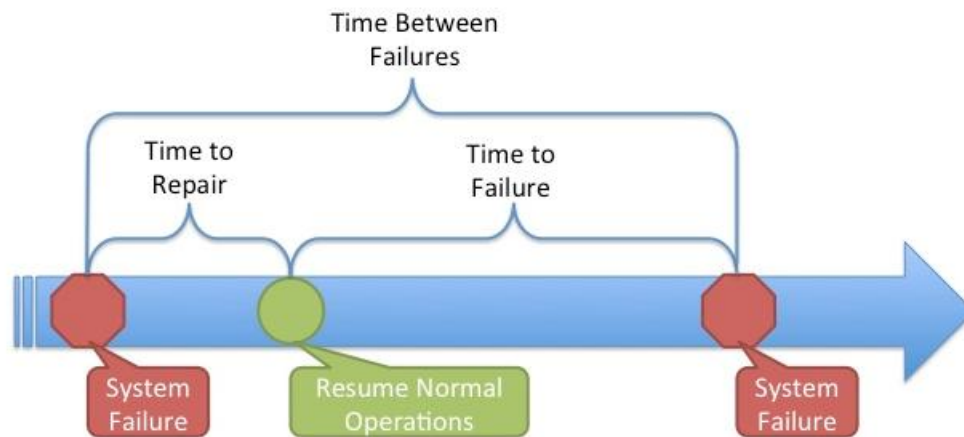
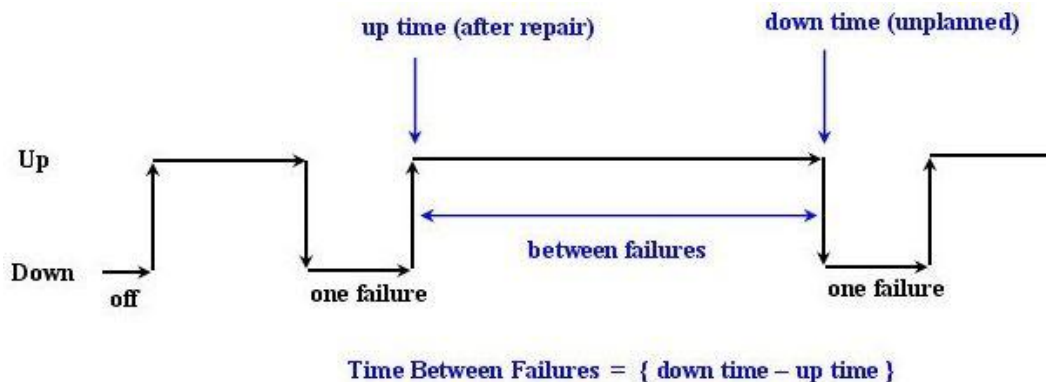


Figura 7 .- Disponibilidad expresada en términos del tiempo hasta el fallo y el tiempo de recuperación ante fallo

La disponibilidad puede conducir a requisitos arquitectónicos, como por ejemplo determinando el número de componentes redundantes (MTBF inferior), la modificabilidad de los componentes (fijando el MTTR inferior) o tipos especiales de componentes (por ejemplo, autodiagnóstico)

Medición: El tiempo medio hasta el fallo y el tiempo medio de reparación de los componentes deben ser identificados (normalmente medidos) o estimados. En este caso el MTTF es igual a (DownTime de un fallo no planificado – “up time” anterior después de reparar/recuperar servicio)



$$\text{Mean time between failures} = \text{MTBF} = \frac{\Sigma(\text{downtime} - \text{uptime})}{\text{number of failures}}$$

Figura 8 .- Tiempo medio entre fallos - MTBF

Un concepto que está estrechamente relacionado con MTBF, y es importante en los cálculos que implican MTBF, es el tiempo medio de “fuera de servicio” o Parada (MDT – Mean Down time). MDT se puede definir como tiempo medio que el sistema está abajo después del fallo. Por lo general, la MDT se considera diferente de MTTR (Mean Time To Repair/recovery); En particular, la MDT suele incluir factores organizativos y logísticos (como los días hábiles o esperar a que los componentes lleguen), mientras que el MTTR suele entenderse como más estrecho y más técnico (teórico).

Esto genera varias aproximaciones o medidas, de lo más ideal a lo más real:

Disponibilidad inherente (Ai): La probabilidad de que un elemento funcione satisfactoriamente en un momento dado cuando se utiliza bajo condiciones establecidas en un entorno de soporte ideal. Excluye tiempo de logística, espera o tiempo de inactividad administrativa, y tiempo de mantenimiento preventivo. Incluye el tiempo de inactividad del mantenimiento correctivo. La disponibilidad inherente se deriva generalmente del análisis de un diseño de ingeniería y se calcula como el tiempo medio

para fallar (MTTF) dividido por el tiempo medio para fallar más el tiempo medio para reparar (MTTR). Se basa en las cantidades bajo control del diseñador.

$$A_i = MTTF / (MTTF + MTTR) = MTTF / MTBF$$

Disponibilidad lograda (Aa): La probabilidad de que un elemento funcione satisfactoriamente en un momento dado cuando se usa bajo condiciones establecidas en un ambiente de soporte ideal (es decir, que el personal, herramientas, repuestos, etc., están disponibles para poder ser usados si se les requiere) Excluye tiempo de logística y espera o tiempo de inactividad administrativa. Incluye tanto mantenimiento preventivo como correctivo.

Disponibilidad operacional (Ao): La probabilidad de que un elemento funcione satisfactoriamente en un momento determinado cuando se utiliza en un entorno operativo y de soporte real o realista. Incluye tiempo de logística, tiempo de preparación y espera o tiempo de inactividad administrativa, y tiempo de mantenimiento preventivo y correctivo. Este valor es igual al tiempo medio entre el fallo (MTBF) dividido por el tiempo medio entre el fallo más el tiempo de parada medio (MDT). Esta medida amplía la definición de disponibilidad a los elementos controlados como la cantidad y la proximidad de repuestos, herramientas y mano de obra.

Análogamente puede haber variaciones del MTBF dependiendo del elemento a considerar:

MTBSA (Mean Time Between System Abort) como tiempo medio entre abortos del sistema o interrupciones controladas del sistema, MTBCF (Mean Time Between Critical Failures) como tiempo medio entre fallos críticos o MTBUR (Mean Time Between Unscheduled Removal) como tiempo medio entre cambios o actualizaciones no programados. Tal nomenclatura se utiliza cuando es deseable diferenciar entre tipos de fallos, tales como fallos críticos y no críticos. Por ejemplo, en un automóvil, el fallo de la radio FM no impide el funcionamiento primario del vehículo. El tiempo medio hasta el fallo (MTTF) se utiliza a veces en lugar de MTBF en los casos en que un sistema se sustituye después de un fallo, ya que MTBF denota el tiempo entre fallos en un sistema que se repara o recupera. MTTFd es una extensión de MTTF, donde MTTFd sólo se preocupa por fallos que podrían resultar peligrosos (Dangerous).

Para medir la fiabilidad y disponibilidad del modelo se pueden utilizar Modelos como el de Markov.

A continuación se relacionan unos cuantos ejemplos de requisitos de disponibilidad:

- ✓ El sistema debe cumplir o exceder el 99.99% de tiempo de actividad.
- ✓ El sistema de indisponibilidad será inferior a 1 hora por 1000 horas de operación.
- ✓ Se necesitan menos de 20 segundos para reiniciar el sistema después de 95% del tiempo. (Este es un requisito MTTR)

Disponibilidad (Availability)	Pérdida de servicio Downtime
90%	36.5 days/year
99%	3.65 days/year
99.9%	8.76 hours/year
99.99%	52 minutes/year
99.999%	5 minutes/year
99.9999%	31 seconds/year

Figura 9 .- Disponibilidad Vs. Tiempo de Pérdida de servicio (Downtime)

2.1.5 Medidas de Usabilidad

En general, se entiende por usabilidad a la facilidad de uso y la formación de los usuarios finales. Este Requisito NF está más desarrollado en el tema específico del temario relativo a la accesibilidad web y usabilidad.

Siguientes medidas más específicas:

- ✓ Capacidad de Aprendizaje: Proporción de funcionalidades o tareas dominadas después de un tiempo de entrenamiento dado
- ✓ Eficiencia
 - ❖ Tiempo de respuesta aceptable
 - ❖ Número de tareas realizadas o problemas resueltos en un tiempo determinado
 - ❖ Número de clics del ratón necesarios para obtener información o funcionalidad
 - ❖ Número de veces que usas el Scroll por pantalla o formulario
- ✓ Capacidad de Memorización: Número (o proporción) de tareas aprendidas que todavía se pueden realizar después de haber dejado de usar el sistema durante un período de tiempo determinado
- ✓ Capacidad para evitar errores o efectos no deseados medidos como:
 - ❖ Número de errores por período de tiempo y clase de usuario
 - ❖ Número de llamadas al soporte de usuarios
- ✓ Manejo de errores: Tiempo medio para recuperarse de un error y poder continuar la tarea
- ✓ Satisfacción del usuario medida como
 - ❖ Ratio de satisfacción por clase de usuario
 - ❖ Ratio de uso de la aplicación, de una funcionalidad, etc..

A continuación se enumeran unos ejemplos de medidas de usabilidad:

- ✓ Cuatro de cada cinco usuarios, después de una introducción de 2 horas al sistema, podrán reservar como huéspedes usando la aplicación en 5 minutos.
- ✓ Los usuarios principiantes deben realizar las tareas X e Y en 15 minutos.
- ✓ Los usuarios experimentados realizarán las tareas X e Y en 2 minutos.
- ✓ Al menos el 80% de los clientes encuestados después de un período de 3 meses deberían tener una satisfacción con el sistema de 7 o más en una escala de 1 a 10.

2.1.6 Medidas de Mantenimiento

Mide la habilidad para realizar cambios de manera rápida y rentable, como por ejemplo:

- ✓ Extensión con nueva funcionalidad
- ✓ Eliminación de capacidades no deseadas
- ✓ Adaptación a nuevos entornos operativos (portabilidad)
- ✓ Reestructuración (racionalización, modularización, optimización, creación de componentes reutilizables)

Está muy relacionado con la "Calidad estática" del Software. Se puede medir en términos de:

- ✓ Métricas de acoplamiento / cohesión, número de anti-patrones, complejidad ciclomática
- ✓ Tiempo medio para corregir un defecto, tiempo medio para agregar nueva funcionalidad
- ✓ Calidad / cantidad de documentación

Algunos de estos requisitos pueden ser medidos con herramientas como SonarQube.

Ejemplos de requisitos de mantenibilidad son:

- ✓ Cada módulo de programa debe ser evaluado para mantenibilidad según el "procedimiento xx". El 70% debe obtener "altamente mantenible" y ninguno "pobre" con la configuración estándar.

- ✓ La complejidad ciclomática del código no debe exceder 7. Ningún método en ningún objeto puede exceder las 200 líneas de código.
- ✓ La instalación de una nueva versión dejará todo el contenido de la Base de datos y la configuración personal sin cambios.
- ✓ El producto deberá proporcionar facilidades para rastrear donde se utiliza cualquier campo de la Base de Datos.

2.1.7 Medidas de Testabilidad

La testabilidad de un sistema mide la capacidad de detectar, aislar y corregir defectos, siendo medidas de referencia las siguientes:

- ✓ Tiempo requerido para ejecutar pruebas
- ✓ Tiempo requerido para configurar el entorno de prueba (desarrollo y ejecución)
- ✓ Probabilidad de fallo visible en presencia de un defecto
- ✓ Cobertura de pruebas (cobertura de requisitos, cobertura de código ...)

Esta característica puede conducir a requisitos arquitectónicos como puede ser la incorporación de mecanismos de seguimiento o la inclusión de puntos de acceso y control adicionales

A continuación se enumeran algunos ejemplos de estos:

- ✓ El sistema suministrado incluirá pruebas de unidad que aseguren una cobertura mínima.
- ✓ El desarrollo debe utilizar pruebas de regresión que permitan un reexamen completo en 12 Horas.
- ✓ Se ejecutarán las pruebas unitarias a través de entorno de Integración continua todos los días a las 01:00.
- ✓ Las pruebas funcionales automatizadas deben realizarse cada vez que se realiza una subida de funcionalidad "mayor".

2.1.8 Medidas de Portabilidad

La portabilidad mide la capacidad del sistema de funcionar bajo diferentes Ambientes o entornos de ejecución, Hardware, software, SO, idiomas, versiones o combinación de estos.

Se puede medir como

- ✓ Número de plataformas específicas (hardware, OS ...).
- ✓ Proporción de componentes o funcionalidad específicos de la plataforma.
- ✓ Tiempo medio de conexión a una plataforma diferente.

A continuación se enumeran algunos ejemplos de estos:

- ✓ No más del 5% de la implementación del sistema será específico para el sistema operativo.
- ✓ El tiempo que se necesita para reemplazar la base de datos relacional actual por otro sistema de base de datos relacional no debe exceder de 2 Horas. No debe producirse pérdida de datos.

2.1.9 Medidas de Integrabilidad y Reutilización

La integrabilidad mide la capacidad de hacer que componentes separados trabajen juntos, pudiéndose expresa en términos del tiempo medio para integrarse con un nuevo sistema de interfaz

La reutilización mide la capacidad de reutilización de los componentes existentes en aplicaciones. Se puede expresar en términos de:

- ✓ Porcentaje de requisitos reutilizados, elementos de diseño, código, pruebas ...
- ✓ Acoplamiento de componentes
- ✓ Grado de utilización de frameworks

2.1.10 Medidas de Robustez

La robustez de un sistema mide la capacidad para hacer frente a lo inesperado.

Se puede medir como

- ✓ Porcentaje de fallos ante entradas no válidas
- ✓ Grado de degradación del servicio
 - ❖ Rendimiento mínimo bajo cargas extremas
 - ❖ Servicios activos en presencia de fallos
 - ❖ Tiempo necesario para gestionar las condiciones de estrés

A continuación se enumeran algunos ejemplos de estos:

- ✓ La pérdida estimada de datos en caso de un fallo de disco será menor de 0,01%.
- ✓ El sistema deberá ser capaz de manejar hasta 10000 usuarios simultáneos. Cuando satisfacen todas sus necesidades y hasta 25000 usuarios concurrentes solamente con capacidades de navegación.

2.1.11 Medidas de Seguridad

En relación a la seguridad se podrían establecer al menos dos medidas:

1. La capacidad de resistir los intentos no autorizados de uso
2. Continuar prestando servicio a los usuarios legítimos mientras se realizan ataque de denegación de servicio (resistencia a los ataques DoS)

Para medir la “calidad” de los requisitos de seguridad podemos contar con los siguientes métodos de medición:

- ✓ Tasa de éxito en la autenticación
- ✓ Resistencia a los ataques conocidos (enumerados)
- ✓ Tiempo / esfuerzos / recursos necesarios para encontrar una clave (probabilidad de encontrar la clave)
- ✓ Probabilidad / tiempo / recursos para detectar un ataque
- ✓ Porcentaje de servicios útiles aún disponibles durante un ataque
- ✓ Porcentaje de ataques con éxito
- ✓ Vida útil de una contraseña, de una sesión
- ✓ Nivel de cifrado exigido según el uso y entorno
- ✓ Mecanismo de bloqueo automáticos por falta de uso de los usuarios

Al igual que con los requisitos de capacidad, los requisitos relativos a la seguridad pueden conducir a requisitos arquitectónicos:

- Autenticación, autorización, auditoría
- Mecanismos de detección
- Firewall, canales de comunicación cifrados
- También puede ser modelado (lógica ...)

A continuación se enumeran algunos ejemplos de requisitos de seguridad

- ✓ La aplicación deberá identificar todas sus aplicaciones cliente antes lo que les permite utilizar sus capacidades.
- ✓ La solicitud deberá asegurar que el nombre del empleado en las bases de datos oficiales de recursos humanos y de nómina coincide exactamente con el nombre impreso en la tarjeta de seguro social del empleado.
- ✓ Al menos el 99% de las intrusiones serán detectadas en 10 segundos.

Se puede tener más información respecto a la seguridad como requisito no funcional en el **tema 120 La seguridad en el nivel de aplicación. Tipos de ataques y protección de servicios web, bases de datos e interfaces de usuario.**

2.1.12 Medidas de privacidad

La privacidad puede entenderse como una parte concreta de la seguridad vinculada a la confidencialidad, centrada en controlar, entre otros aspectos, que no haya accesos indebidos. Un ejemplo puede ser, por ejemplo, que en un banco se hagan auditorías de si sus empleados han accedido a las cuentas solo cuando les correspondía y no por cotillear, o en el caso de la Plataforma de Intermediación de datos que las consultas de datos personales están justificadas con el consentimiento del ciudadano para un procedimiento o servicio concreto, o un expediente iniciado de oficio en el caso de que la consulta se pueda realizar sin dicho consentimiento amparada por una Ley que exige de recabarlos.

Las medidas para asegurar la privacidad en aplicaciones o usos convencionales están ligadas a las técnicas de control de acceso y las auditorías asociadas a las mismas.

Privacidad y servicios basados en la localización

Cada vez más, los dispositivos móviles ofrecen servicios basados en la localización y facilitan el seguimiento de la misma. Esto crea problemas de privacidad del usuario.

La ubicación y las preferencias de un usuario constituyen información personal. Su uso incorrecto viola la privacidad de ese usuario.

Un estudio de marzo de 2013 del MIT ("**Unique in the Crowd: The privacy bounds of human mobility**") mostró que 4 puntos espacio-temporales, lugares y tiempos aproximados, son suficientes para identificar de manera única el 95% de 1,5 millones de personas en una base de datos de movilidad. El estudio muestra además que estas restricciones se mantienen incluso cuando la resolución del conjunto de datos es baja. Por lo tanto, incluso los datos gruesos o borrosos proporcionan poco anonimato.

Se han propuesto varios métodos para proteger la privacidad de los usuarios en servicios basados en la localización, incluyendo el uso de servidores anónimos, la difuminación de información, etc. También se han propuesto métodos para cuantificar la privacidad y calcular el equilibrio entre el beneficio de proporcionar información de localización precisa y los inconvenientes de arriesgar la privacidad personal. Los usuarios de estos servicios también pueden optar por mostrar información de ubicación más genérica (por ejemplo, "En la ciudad" o "En el Trabajo") a algunos de sus conocidos más casuales, mientras que sólo muestran información específica de ubicación, como su dirección exacta a contactos más cercanos como cónyuge, familiares y buenos amigos.

- ✓ **Privacidad por diseño:** El principio de privacidad por diseño establece que la privacidad y la protección de datos están integradas a lo largo de todo el ciclo de vida de las tecnologías, desde la etapa de diseño inicial hasta su despliegue, uso y eliminación final.
- ✓ **Ingeniería de privacidad:** La práctica de construir, aparentemente, software o sistemas de información que se adhieren a las políticas de privacidad y conformidades pertinentes es un área en desarrollo y se conoce como ingeniería de privacidad
- ✓ **Auto-sincronización de la privacidad:** La auto-sincronización de la privacidad es el modo mediante el cual las partes interesadas de un programa de privacidad empresarial contribuyen espontáneamente, de manera colaborativa al éxito máximo del programa. Las partes interesadas (stakeholders) pueden ser clientes, empleados, gerentes, ejecutivos, proveedores, socios o inversionistas. Cuando se alcanza la auto-sincronización, el modelo establece que los intereses personales de los individuos hacia su privacidad están en equilibrio con los intereses comerciales de las empresas que recogen y usan la información personal de esos individuos.

2.1.13 Otros requisitos no funcionales

Un aspecto importante a tener en cuenta en el desarrollo de sistemas son todas esas cualidades que puede demandar el negocio, relativas a la Experiencia de Usuario, y que pueden ser vitales para el éxito como "divertido" o "genial" o "hermoso" o "emocionante". Obviamente no son una necesidad

“funcional” de la aplicación o del sistema pero pueden llegar a ser más importante que la mayor parte de requisitos. ¿Cómo se pueden medir?

Es necesario aplica mecanismos parecidos a los que hemos visto respecto a la usabilidad, pero teniendo en cuenta que podemos estar delante de elementos completamente subjetivos de difícil cuantificación, salvo encuestas de opinión o sesiones de trabajo con usuarios “objetivo” del sistema.

No debemos olvidar señalar la importancia de los requisitos legales, por ejemplo en el caso de la Accesibilidad para un Sistema Web marcaría unos grados de accesibilidad concretos, o en el caso de aplicaciones de Videojuegos el cumplimiento con los sistemas de clasificación por edad / contenido (por ejemplo, sin violencia).

Otro aspecto a tener en cuenta es que las listas de atributos de calidad existentes son interesantes y fundamentales, pero no incluyen todos los RNF.

A veces es mejor dejar que los clientes hagan brainstorming sobre sus necesidades e intereses antes de proponer exclusivamente las categorías convencionales de RNF.

En cualquier caso, hay que ser consciente de que se deben refinar los objetivos y metas del sistema en requisitos medibles.

Lograr clasificar adecuadamente cada requisito no funcional identificado es muy importante para garantizar este proceso.

3 Estándares relativos a Requisitos.

En relación al proceso de análisis y gestión de requisito, existen numerosas metodologías, estándares y normas. Entre las normas más conocidas destacan las normas **ISO 29148**, **ISO 15288**, **ISO 24766** e **ISO/IEC 25000**.

ISO 29148 – Systems and software engineering – Life cycle processes – Requirements engineering: Contiene técnicas para cubrir los procesos y productos relacionados con la ingeniería de requisitos a lo largo de todo el ciclo de vida.

- ✓ Define la construcción de un buen requisito, proporciona atributos y características de los requisitos y analiza la aplicación iterativa y recursiva de los procesos de requisitos a lo largo del ciclo de vida.
- ✓ Proporciona una orientación adicional en la aplicación de los procesos de requisitos de ingeniería y gestión de las actividades de los requisitos relacionados en la norma ISO 15288.

ISO 15288 - Systems and software engineering – System life cycle processes: Establece un marco común de procesos para describir el ciclo de vida de la Ingeniería de Sistemas.

- ✓ Define un conjunto de procesos y la terminología asociada desde un punto de vista de la ingeniería. Estos procesos se pueden aplicar en cualquier nivel de la estructura jerárquica de un sistema y se pueden aplicar en todo el ciclo de vida de la gestión y la realización de las etapas del ciclo de vida de un sistema fomentando la participación de todas las partes interesadas con el objetivo satisfacer las necesidades del cliente.

ISO 24766 – Information technology – Systems and software engineering – Guide for requirements tool capabilities:

- ✓ Esta norma proporciona una orientación sobre las capacidades deseables que debería aportar una herramienta de Ingeniería de Requisitos. La norma ISO 24766 es utilizada habitualmente como complemento de la norma ISO 14102, “Information technology – Guideline for the evaluation and selection of CASE tools”

Otras normas y estándares sobre la gestión de requisitos son:

- ✓ IEEE Std 830 – IEEE Recommended Practice for Software Requirements Specifications
- ✓ IEEE Std 1233 – IEEE Guide for Developing System Requirements Specifications

ISO/IEC 25000: El objetivo general de la creación del estándar ISO/IEC 25000 SQuaRE (System and Software Quality Requirements and Evaluation) es organizar, enriquecer y unificar las series que cubren dos procesos principales: especificación de requisitos de calidad del software y evaluación de la calidad del software, soportada por el proceso de medición de calidad del software.

Las características de calidad y sus mediciones asociadas pueden ser útiles no solamente para evaluar el producto software sino también para definir los requerimientos de calidad. La serie ISO/IEC 25000:2005 reemplaza a dos estándares relacionados: ISO/IEC 9126 (Software Product Quality) e ISO/IEC 14598 (Software Product Evaluation)

Un factor clave para las organizaciones es la cobertura de la herramienta (o herramientas en el caso de usar varias) de gestión de requisitos que se vaya a utilizar en la organización en referencia a las normas y estándares anteriores.

3.1 ISO/IEC 25010

El modelo de calidad representa la piedra angular en torno a la cual se establece el sistema para la evaluación de la calidad del producto. En este modelo se determinan las características de calidad que se van a tener en cuenta a la hora de evaluar las propiedades de un producto software determinado.

La calidad del producto software se puede interpretar como el grado en que dicho producto satisface los requisitos de sus usuarios aportando de esta manera un valor. Son precisamente estos requisitos (funcionalidad, rendimiento, seguridad, mantenibilidad, etc.) los que se encuentran representados en el modelo de calidad, el cual categoriza la calidad del producto en características y subcaracterísticas.

El modelo de calidad del producto definido por la ISO/IEC 25010 se encuentra compuesto por las ocho características de calidad:

- ✓ Adecuación Funcional
- ✓ Eficiencia de desempeño
- ✓ Compatibilidad
- ✓ Usabilidad
- ✓ Fiabilidad
- ✓ Seguridad
- ✓ Mantenibilidad
- ✓ Portabilidad

que se muestran expandidas en la siguiente figura y que se describe a continuación tal y como consta en la web iso25000.com.



Figura 10 .- Modelo de calidad ISO/IEC 25010

3.1.1 Adecuación Funcional

Representa la capacidad del producto software para proporcionar funciones que satisfacen las necesidades declaradas e implícitas, cuando el producto se usa en las condiciones especificadas. Esta característica se subdivide a su vez en las siguientes subcaracterísticas:

- ✓ Completitud funcional. Grado en el cual el conjunto de funcionalidades cubre todas las tareas y los objetivos del usuario especificados.
- ✓ Corrección funcional. Capacidad del producto o sistema para proveer resultados correctos con el nivel de precisión requerido.
- ✓ Pertinencia funcional. Capacidad del producto software para proporcionar un conjunto apropiado de funciones para tareas y objetivos de usuario especificados.

Esta característica se corresponde con los requisitos funcionales cubiertos en el tema 84. Análisis de aspectos funcionales del temario.

3.1.2 Eficiencia de desempeño

Esta característica representa el desempeño relativo a la cantidad de recursos utilizados bajo determinadas condiciones. Esta característica se subdivide a su vez en las siguientes subcaracterísticas:

- ✓ Comportamiento temporal. Los tiempos de respuesta y procesamiento y los ratios de throughput de un sistema cuando lleva a cabo sus funciones bajo condiciones determinadas en relación con un banco de pruebas (benchmark) establecido.
- ✓ Utilización de recursos. Las cantidades y tipos de recursos utilizados cuando el software lleva a cabo su función bajo condiciones determinadas.
- ✓ Capacidad. Grado en que los límites máximos de un parámetro de un producto o sistema software cumplen con los requisitos.

3.1.3 Compatibilidad

Capacidad de dos o más sistemas o componentes para intercambiar información y/o llevar a cabo sus funciones requeridas cuando comparten el mismo entorno hardware o software. Esta característica se subdivide a su vez en las siguientes subcaracterísticas:

- ✓ Coexistencia. Capacidad del producto para coexistir con otro software independiente, en un entorno común, compartiendo recursos comunes sin detrimento.
- ✓ Interoperabilidad. Capacidad de dos o más sistemas o componentes para intercambiar información y utilizar la información intercambiada.

3.1.4 Usabilidad

Capacidad del producto software para ser entendido, aprendido, usado y resultar atractivo para el usuario, cuando se usa bajo determinadas condiciones. Esta característica se subdivide a su vez en las siguientes subcaracterísticas:

- ✓ Capacidad para reconocer su adecuación. Capacidad del producto que permite al usuario entender si el software es adecuado para sus necesidades.
- ✓ Capacidad de aprendizaje. Capacidad del producto que permite al usuario aprender su aplicación.
- ✓ Capacidad para ser usado. Capacidad del producto que permite al usuario operarlo y controlarlo con facilidad.
- ✓ Protección contra errores de usuario. Capacidad del sistema para proteger a los usuarios de hacer errores.
- ✓ Estética de la interfaz de usuario. Capacidad de la interfaz de usuario de agradar y satisfacer la interacción con el usuario.
- ✓ Accesibilidad. Capacidad del producto que permite que sea utilizado por usuarios con determinadas características y discapacidades.

3.1.5 Fiabilidad

Capacidad de un sistema o componente para desempeñar las funciones especificadas, cuando se usa bajo unas condiciones y periodo de tiempo determinados. Esta característica se subdivide a su vez en las siguientes subcaracterísticas:

- ✓ Madurez. Capacidad del sistema para satisfacer las necesidades de fiabilidad en condiciones normales.
- ✓ Disponibilidad. Capacidad del sistema o componente de estar operativo y accesible para su uso cuando se requiere.
- ✓ Tolerancia a fallos. Capacidad del sistema o componente para operar según lo previsto en presencia de fallos hardware o software.
- ✓ Capacidad de recuperación. Capacidad del producto software para recuperar los datos directamente afectados y reestablecer el estado deseado del sistema en caso de interrupción o fallo.

3.1.6 Seguridad

Capacidad de protección de la información y los datos de manera que personas o sistemas no autorizados no puedan leerlos o modificarlos. Esta característica se subdivide a su vez en las siguientes subcaracterísticas:

- ✓ Confidencialidad. Capacidad de protección contra el acceso de datos e información no autorizados, ya sea accidental o deliberadamente.
- ✓ Integridad. Capacidad del sistema o componente para prevenir accesos o modificaciones no autorizados a datos o programas de ordenador.
- ✓ No repudio. Capacidad de demostrar las acciones o eventos que han tenido lugar, de manera que dichas acciones o eventos no puedan ser repudiados posteriormente.
- ✓ Responsabilidad. Capacidad de rastrear de forma inequívoca las acciones de una entidad.

- ✓ Autenticidad. Capacidad de demostrar la identidad de un sujeto o un recurso.

3.1.7 Mantenibilidad

Esta característica representa la capacidad del producto software para ser modificado efectiva y eficientemente, debido a necesidades evolutivas, correctivas o perfectivas. Esta característica se subdivide a su vez en las siguientes subcaracterísticas:

- ✓ Modularidad. Capacidad de un sistema o programa de ordenador (compuesto de componentes discretos) que permite que un cambio en un componente tenga un impacto mínimo en los demás.
- ✓ Reusabilidad. Capacidad de un activo que permite que sea utilizado en más de un sistema software o en la construcción de otros activos.
- ✓ Analizabilidad. Facilidad con la que se puede evaluar el impacto de un determinado cambio sobre el resto del software, diagnosticar las deficiencias o causas de fallos en el software, o identificar las partes a modificar.
- ✓ Capacidad para ser modificado. Capacidad del producto que permite que sea modificado de forma efectiva y eficiente sin introducir defectos o degradar el desempeño.
- ✓ Capacidad para ser probado. Facilidad con la que se pueden establecer criterios de prueba para un sistema o componente y con la que se pueden llevar a cabo las pruebas para determinar si se cumplen dichos criterios.

3.1.8 Portabilidad

Capacidad del producto o componente de ser transferido de forma efectiva y eficiente de un entorno hardware, software, operacional o de utilización a otro. Esta característica se subdivide a su vez en las siguientes subcaracterísticas:

- ✓ Adaptabilidad. Capacidad del producto que le permite ser adaptado de forma efectiva y eficiente a diferentes entornos determinados de hardware, software, operacionales o de uso.
- ✓ Capacidad para ser instalado. Facilidad con la que el producto se puede instalar y/o desinstalar de forma exitosa en un determinado entorno.
- ✓ Capacidad para ser reemplazado. Capacidad del producto para ser utilizado en lugar de otro producto software determinado con el mismo propósito y en el mismo entorno.

4 Ejemplo de Documento de Requisitos NF - SGAD

A continuación se incluye un ejemplo parcial de Documento de Licitación, basado en Concurso AM/26, relativo a requisitos no funcionales.

Descripción de los requisitos funcionales (No se incluye en el ejemplo)

Descripción de los requisitos no funcionales

- 1) *Gestión*
- 2) *Seguimiento y Control*
- 3) *Calidad de los desarrollos*
- 4) *Seguridad*
- 5) *Sistemas*
- 6) *Diseño*
- 7) *Entregables*

DESCRIPCIÓN DE LOS REQUISITOS NO FUNCIONALES

Gestión

GES 1. Se deberá disponer de una herramienta para el control de los Elementos de Configuración.

Seguimiento y Control

SC 1. Se realizarán reuniones de seguimiento con el objetivo de:

- Realizar un seguimiento del contrato,
- Aprobar los productos finales y tomar las decisiones de mayor importancia en el ciclo de vida del desarrollo.
- Verificar el grado de cumplimiento de los objetivos
- Verificar la calidad del servicio ofrecido por el adjudicatario
- Revisar las reasignaciones y variaciones del personal dedicado al proyecto
- Validar la planificación de las actividades realizadas

SC 2. Tras las reuniones de seguimiento el MINHAFP podrá rechazar en todo o en parte los trabajos realizados en la medida que correspondan a los especificados o no superasen los controles de calidad acordados.

SC 3. Cada reunión tendrá como entrada mínima:

- Agenda de la reunión
- Informe de progreso y actividades.
- Principales incidencias ocurridas
- Lista de acciones pendientes
- Planificación actualizada.

SC 4. En el caso de que se produzcan eventualidades que hagan variar la planificación o la organización del proyecto, deberán notificarse en una reunión de seguimiento, y será la Administración quien autorice las soluciones más adecuadas, el adjudicatario no podrá realizar ninguna variación sin la autorización expresa de ésta.

SC 5. El MINHAFP se encargará de coordinar las reuniones que fuera necesario realizar con las unidades que colaboran en el proyecto.

Calidad de los desarrollos

CD 1. El adjudicatario entregará un informe de la herramienta Sonar para mostrar que el código sigue los patrones y directrices de codificación y diseño que se definan para el proyecto.

CD 2. Deberá adjuntarse un informe explicativo siguiendo el modelo que proponga el MINHAFP, analizando los resultados obtenidos y comparándolo con los parámetros aceptados de calidad.

CD 3. El adjudicatario se compromete a utilizar para sus trabajos el entorno de desarrollo integrado proporcionado por el MINHAFP, tanto en cuanto a repositorios de código y herramientas, como repositorios de documentación. El entorno se encuentra en constante evolución, en este momento las herramientas disponibles son:

- Sistema de control de versiones software: Subversion, opcionalmente GIT
- Gestión y construcción de artefactos: Maven 3
- Repositorio de artefactos: Artifactory, opcionalmente Nexus o Archiva
- Sistema de generación automática: Jenkins
- Sistema de control de calidad: SonarQube, PMDFindbugs
- Sistema de gestión de incidencias: Mantis Debbbug tracker (Opcionalmente JIRA)

A las propias herramientas de desarrollo, se unen otras que complementan la gestión de los proyectos:

- Documentación de Proyectos: Alfresco
- Sistema de gestión de pruebas: TestLink
- Planificación de Proyectos: DotProject (opcionalmente Redmine)

Otros productos auxiliares para testing y evaluación:

- SOAP-UI
- Selenium
- Jmeter

Seguridad

SEG 1. Se tendrán en cuenta en todo momento los requisitos de seguridad establecidos por el Esquema Nacional de Seguridad.

SEG 2. Se definirá e implementará un mecanismo de gestión, creación y administración segura de Logs.

Sistemas

SIS 1. El MINHAFP dispone de los entornos de explotación y pre-explotación específicos donde habrá de implementarse la solución con la siguiente infraestructura:

- Lenguaje y tecnologías de desarrollo: Java, XML, SOAP, HTML.
- Plataforma Hardware/SW
 - **Servidores de aplicaciones Intel o similar con balanceo de carga.**
 - **Sistema Operativo: Linux**
- Software de los servidor de aplicaciones:
 - **Tomcat. (Incluirá todos los de la matriz de compatibilidad del portfolio SCSPv3)**
 - **Apache (opcional)**
- Bases de Datos
 - **Oracle 11g (O superior) , Oracle RAC, Maria DB, Mysql, SQLServer.**

Diseño

DIS 1. El adjudicatario realizará una Especificación de Requisitos del Sistema (ERS) previo al inicio del diseño y desarrollo de los componentes y APIs.

DIS 2. En el diseño software se seguirán las siguientes premisas:

- Se elegirá una arquitectura que minimice el número de interfaces entre los diferentes componentes y de forma que se garantice la rapidez, seguridad, facilidad y economía en la operación y mantenimiento del sistema.
 - Se utilizará una arquitectura basada en componentes, que permita una alta cohesión y un bajo acoplamiento.
 - Se seguirán los principios de modularidad que faciliten la reutilización del código, resolución de errores, mantenimiento y evolución tecnológica de la solución.
- DIS 3. Las librerías, componentes, APIs, o cualquier software externo que se utilice en los desarrollo será compatible con la licencia de software libre del sistema, de forma que el código desarrollado pueda ser liberado con la misma licencia.
- DIS 4. El adjudicatario se compromete a utilizar el repositorio de código (SVN o GIT) que la administración contratante disponga y a utilizar la herramienta Maven para organizar el código fuente. El uso de Maven ha de permitir generar todos los ejecutables y/o librerías, y empaquetar y desplegar el resultado de forma sencilla y rápida.
- DIS 5. El adjudicatario diseñará y programará las pruebas unitarias y de integración para realizarse de manera automática, preferiblemente con la herramienta JUnit. En aquellos casos donde JUnit no permita verificar el funcionamiento correcto del código, se diseñarán las pruebas con una herramienta alternativa apropiada.
- DIS 6. Diseño de los casos de prueba y ejecución de los mismos para la consulta de transmisiones de datos Personales.

Entregables

- ENT 1. Toda la documentación se entregará al menos en español, siguiendo las plantillas de la organización y en el soporte que se acuerde para facilitar el tratamiento y reproducción de los mismos.
- ENT 2. La documentación que se deberá aportar, será al menos, la indicada en los requisitos de diseño y en los requisitos siguientes, con los mínimos asociados a los documentos referidos.

Se entregará la siguiente documentación para la parte de Verificación y Certificación de servicios SCSPv3:

- Casos de pruebas definidos para probar cada servicio ofrecido. El número de casos de prueba mínimo será de 30 casos significativos por servicio. En caso de no disponer de suficientes datos de prueba este número de casos podrá ser inferior, siempre que cubra toda la casuística posible.
 - Relación de las incidencias detectadas y propuesta de solución
 - Interfaz/Servicio
 - Negocio/documentación
- ENT 3. El contratista realizará documentos de diseño de todo el software que forme parte del sistema y de cada uno de los subsistemas y componentes
- Arquitectura software.
 - Descripción funcional de los módulos.
 - Análisis funcional: abarcando como mínimo los distintos módulos de la plataforma en cuestión y un nivel de detalle suficiente para permitir la extensión o manipulación.
 - Diseño técnico: complementando el análisis funcional y representando las distintas conexiones entre los distintos elementos o módulos.
- ENT 4. El adjudicatario proporcionará Manuales de Usuario/Administrador e Integrador que describan la instalación, configuración y operación del sistema como conjunto y de cada uno de los subsistemas y componentes, incluyendo el hardware, si aplica.

Los manuales comprenderán tres tipologías:

- Manual de instalación y despliegue. Comprende un manual procedimental del proceso de instalación y/o despliegue de las correspondientes aplicaciones. Además se deberá complementar con un manual de instalación segura, que incluya recomendaciones o buenas prácticas a llevar a cabo en el despliegue de la citada aplicación.
- Manual de usuario o administrador. Manual que muestre las funcionalidades proporcionadas por el panel de administrador de la plataforma en cuestión. Al igual que en el caso anterior, se incluirá un apéndice que señale claramente las buenas prácticas a nivel de administración segura.
- Manual de integrador. Diseñado para proporcionar a las aplicaciones clientes. Contendrá toda la información necesaria para integrar los servicios proporcionados, así como referencias a estándares empleados.

ENT 5. Documentación de código con Javadoc

Además se deberá incluir la correspondiente documentación de código en los lenguajes y para los componentes objeto del documento de licitación con los niveles de cobertura siguientes:

- Cobertura básica o de interfaces públicas. Es la documentación necesaria para realizar una correcta integración de las interfaces. Comprende los métodos, variables, parámetros, clases y constantes públicas que podrían ser requeridas por integradores. Se considerará como un nivel básico de documentación y se incluirá como parte del kit de integración particular. Se requiere el 100% de los elementos detectados como de integración (capas de ws, servlets, etc).
- Cobertura de clases y paquetes. Se deberá dotar de un javadoc explicando la utilidad de las clases y paquetes a todo el código. Dentro de este cabe destacar la especificación de una cabecera predefinida para todas las clases y paquetes, así como un acuerdo de licencia. Se requiere el 100% del código.

ENT 6. El adjudicatario presentará un Plan de Pruebas del Sistema en el que se incluya al menos:

- Diseño de las pruebas que garantice la cobertura de los requisitos definidos, incluyendo pruebas de campo que validen los requisitos de usuario. En el diseño de las pruebas se mostrará la trazabilidad de las pruebas de requisitos con las pruebas unitarias y de integración automatizadas correspondientes.
- Procedimientos de Pruebas en el que se desarrollarán las pruebas diseñadas.
- Informe de Pruebas en el que se plasmarán los resultados e incidencias ocurridas. Todas las pruebas diseñadas en el Plan de Pruebas del Sistema incluirán sus respectivos criterios de validación que incluirán las funcionalidades básicas y los parámetros de rendimiento más importantes.

ENT 7. El adjudicatario entregará en el Hito Final todo el código fuente, los ejecutables, librerías, asistentes de instalación de todo el sistema y las licencias necesarias (incluidas las de software libre que se use), correspondientes al sistema completo.

ENT 8. Los entregables del contrato debidamente actualizados a su última versión, se entregará como una unidad en el hito final del proyecto, independientemente de las entregas de las líneas base iniciales o versiones a lo largo del proyecto.

ENT 9. Todos los entregables documentales del contrato se entregarán en un formato electrónico editable y compatible con los formatos utilizados por la administración contratante.

ENT 10. La propiedad de la documentación y de los trabajos realizados será exclusiva del MINHAFP, sin que el contratista pueda conservarla, ni obtener copia de la misma.

5 Glosario

DoS: Denial of Service / Denegación de servicio

FURPS+: Functionality, Usability, Reliability, Performance, Supportability

ISO/IEC 25010: Estándar ISO/IEC 25000 SQuaRE (System and Software Quality Requirements and Evaluation), Calidad del producto SW

MAGERIT: Metodología de Análisis y riesgos de los Sistemas de Información

MIT: Instituto tecnológico de Massachusetts.

MDT: Tiempo medio de “fuera de servicio” o Parada (MDT).

MTBCF: Tiempo medio entre fallos críticos

MTBF: Tiempo medio entre fallos

MTBSA: Tiempo medio entre abortos del sistema

MTBUR: Tiempo medio entre cambios o actualizaciones no programados.

MTTF: Tiempo medio hasta el fallo, equivale al tiempo de servicio

MTTFd: Tiempo medio hasta el fallo peligroso

MTTR: Mean Time To Repair/recovery

NTI's: Normas Técnicas de Interoperabilidad

OTAN: Organización del Tratado del Atlántico Norte

RGPD-UE: Reglamento General de Protección de Datos de la Unión Europea

RNF: Requisitos no funcionales

SNC: criterios de Seguridad, Normalización y Conservación

SPD: seguridad del proceso de desarrollo

SSI: seguridad del sistema de información

SVN: sistema de control de versiones software Subversion

TCO: coste total de propiedad

CONCLUSIONES

La amplitud y las aristas del tema son muy amplias. El contenido está muy relacionado tanto con los requisitos funcionales que determinarán los requisitos no funcionales, como con las métricas de calidad que permitirán estimar el cumplimiento de los mismos.

Elementos clave, recordar la importancia de la clasificación y estructuración de los mismos, destacando la clasificación de Ian Sommerville: requisitos se clasifican de acuerdo con sus implicaciones en Producto, Organizacionales y Externos.

Otra clasificación muy utilizada es la **FURPS+**: que hace referencia a Functionality, Usability, Reliability, Performance, Supportability, y el + se añadió posteriormente al modelo después de varias campañas en HP para extender el acrónimo para enfatizar e identificar categorías adicionales que generalmente representan restricciones relativos a Diseño, Implementación, Interfaz y Requisitos físicos.

Los requisitos no funcionales se asocian tradicionalmente a "**atributos de calidad**" de un sistema. Otros términos para requisitos no funcionales son "**cualidades**", "**objetivos de calidad**", "**requisitos de calidad de servicio**", "**limitaciones**" y "**requisitos no relacionados con el comportamiento**". Informalmente, en Inglés se refiere a ellos como "-ilities" como la estabilidad (stability) o la portabilidad (portability).

Es por eso que en el tema se vinculan a las medidas de los mismos como referencias a lo bueno o malo que es un sistema, en el sentido de si cumple o no con los umbrales definidos.

Entre los requisitos no funcionales destacan:

- ✓ **Coste:** coste total de propiedad (TCO) para adquisición, instalación, uso, disposición
- ✓ **Disponibilidad:** Corresponde al tiempo total en que un sistema puede ser usado en un período determinado. También puede definirse el grado en que un sistema está en un estado operable definido cada vez que se necesite.
- ✓ **Eficiencia:** uso mínimo de recursos (memoria, procesador, disco, red...)
- ✓ **Escalabilidad:** para un gran número de usuarios o cantidades de datos o incrementos de las cifras de rendimiento actuales.
- ✓ **Escalabilidad:** Capacidad de un sistema o servicio de TI de manejar una creciente carga de trabajo, por ejemplo mayor número de conexiones o usuarios. La escalabilidad puede ser horizontal (más Equipos) o vertical (más capacidad de los equipos)
- ✓ **Extensibilidad:** Grado en que la implementación del sistema toma en consideración y facilita su crecimiento en el futuro.
- ✓ **Mantenibilidad:** Mide la facilidad con que puede darse mantenimiento al producto (en este caso al software o servicio de TI), con la finalidad de: desarrollar nuevos requisitos/funcionalidades, aislar los defectos y sus causas, corregir estos defectos y atender las demandas del entorno cambiante.
- ✓ **Rendimiento:** Tasa de producción (Throughput) o la velocidad (Tiempo de respuesta/Speed) a la que se puede procesar algo. Se mide en cada caso como "Unidades/peticiones por unidad de tiempo" generalmente en segundos o como tiempo de respuesta respectivamente
- ✓ **Resiliencia:** Es la capacidad de absorber o evitar daños sin sufrir un fallo completo. Capacidad de proporcionar y mantener un nivel aceptable de servicio ante la ocurrencia de fallos y desafíos a la operación normal. Nos referimos a cyber-resiliencia ante ataques cibernéticos.
- ✓ **Robustez:** Capacidad de funcionar en presencia de fallos, tensiones, entradas inválidas ...
- ✓ **Seguridad:** Grado de protección de los datos, software y plataforma de tecnología de posibles pérdidas, actividades no permitidas o uso para propósitos no establecidos previamente. También relativa a la seguridad de uso. (Seguridad industrial)
- ✓ **Usabilidad:** Definido como la facilidad de uso y aprendizaje de un Sistema, Software o Servicio de Tecnología de Información. esfuerzo requerido para aprender, usar, aportar e interpretar Resultados de un programa

De ellos, los relativos al producto con carácter técnico se pueden medir y monitorizar mediante herramientas automatizadas.

PREGUNTAS DE TEST

- 1) **El modelo de calidad del producto definido por la ISO/IEC 25010 se encuentra compuesto por las ocho características de calidad, señale la que no lo sea:**
 - a) Adecuación Funcional
 - b) Rendimiento
 - c) Compatibilidad
 - d) Usabilidad

- 2) **El modelo de calidad del producto definido por la ISO/IEC 25010 se encuentra compuesto por las ocho características de calidad, señale la que no lo sea:**
 - a) Fiabilidad
 - b) Seguridad
 - c) Mantenibilidad
 - d) Todas las anteriores lo son

- 3) **De los siguientes ámbitos de clasificación de requisitos no funcionales propuestos por Sommerville, señale los que sean**
 - a) Producto, Organizacionales y Externos
 - b) Negocio, Técnicos y de Calidad
 - c) Negocio, Externos y de Calidad
 - d) Ninguno de los anteriores

- 4) **FURPS es un acrónimo que representa:**
 - a) Funcionalidad, Usabilidad, Fiabilidad, Rendimiento y Capacidad de soporte
 - b) Funcionalidad, Usabilidad, Rendimiento, Portabilidad y Capacidad de soporte
 - c) Funcionalidad, Usabilidad, Resiliencia, Portabilidad y Sostenibilidad
 - d) Ninguna de las anteriores

- 5) **La eliminación de capacidades no deseadas es un ejemplo de medida relacionada con el RNF:**
 - a) Usabilidad
 - b) Mantenibilidad
 - c) Accesibilidad
 - d) Disponibilidad

- 6) **El sistema no estará indisponible más de 1 hora por 1000 horas de operación es un ejemplo de medida relacionada con:**
 - a) Usabilidad
 - b) Mantenibilidad
 - c) Accesibilidad
 - d) Disponibilidad

- 7) **No más de 1 por 1.000.000 transacciones resultará en un fallo requiriendo un reinicio del sistema es un ejemplo de medida relacionada con:**
 - a) Fiabilidad
 - b) Mantenibilidad
 - c) Disponibilidad
 - d) Ninguna de las anteriores

- 8) **La fiabilidad se puede medir usando las siguientes medidas:**
 - a) Probabilidad de que el sistema realice su función requerida para un intervalo bajo condiciones establecidas
 - b) MTBF: Tiempo medio transcurrido entre los fallos

- c) Tasa de defectos
- d) Todas las anteriores lo son

9) La pérdida estimada de datos en caso de un fallo de disco será menor de 0,01%.

- a) Robustez
- b) Integrabilidad
- c) Fiabilidad
- d) Reutilización

10) Señale cual es una medida de usabilidad

- a) Tiempo de respuesta aceptable
- b) Número de tareas realizadas o problemas resueltos en un tiempo determinado
- c) Número de clics del ratón necesarios para obtener información o funcionalidad
- d) Todas las anteriores

11) La disponibilidad de un sistema se mide como:

- a) Disponibilidad = $MTBF / (MTBF + MTTR)$
- b) Disponibilidad = $MTBF + MTTR$
- c) Disponibilidad = $MTBF - MTTR$
- d) Ninguna de las anteriores

12) Si la Disponibilidad es del 99.9% significa que la pérdida de servicio será inferior a

- a) 36.5 días / año
- b) 3.65 días / año
- c) 8.76 horas / año
- d) 52 minutos / año

13) Según la ISO/IEC 25010, la característica de usabilidad se subdivide en las siguientes subcaracterísticas. Señale la que no lo sea.

- a) Protección contra errores de usuario. Capacidad del sistema para proteger a los usuarios de hacer errores.
- b) Estética de la interfaz de usuario. Capacidad de la interfaz de usuario de agradar y satisfacer la interacción con el usuario.
- c) Accesibilidad. Capacidad del producto que permite que sea utilizado por usuarios con determinadas características y discapacidades.
- d) Todas las anteriores lo son

14) Respecto a la fiabilidad señale la afirmación correcta:

- a) La Fiabilidad es la capacidad de un sistema o componente para desempeñar las funciones especificadas.
- b) La Fiabilidad es la capacidad de un sistema o componente para desempeñar las funciones especificadas, cuando se usa bajo unas condiciones acordadas.
- c) La Fiabilidad es la capacidad de un sistema o componente para desempeñar las funciones especificadas, cuando se usa bajo unas condiciones y periodo de tiempo determinados.
- d) Ninguna de las definiciones anteriores es completa o correcta

15) La paradoja de la privacidad, es conocida por el hecho de que:

- a) Los usuarios no se preocupan por su seguridad porque asumen que el sistema lo hará
- b) Los usuarios se preocupan por su seguridad pero publican información suya sin control en las redes sociales
- c) Los usuarios se preocupan por su seguridad aunque no sería necesario porque el sistema ya lo hace perfectamente
- d) Ninguna de las anteriores

SOLUCIONES A LAS PREGUNTAS DE TEST

PREGUNTA	SOLUCIÓN
1	B
2	D
3	A
4	A
5	B
6	D
7	A
8	D
9	A
10	D
11	A
12	C
13	D
14	C
15	B

BIBLIOGRAFÍA BÁSICA

1. Sommerville, Ian. Ingeniería del Software. Ed. Addison Wesley. 9ª Ed. 2011. ISBN: 9780137035151
2. Metodología Métrica versión 3. Ministerio de Administraciones Públicas. 1998. http://administracionelectronica.gob.es/pae/Home/pae_Documentacion/pae_Metodolog/pae_Metrica_v3.html
3. Pressman, Roger. Ingeniería del Software. Un Enfoque Práctico. Ed. McGraw Hill. 6ª Ed. 2005. ISBN 970-105473-3
4. Donald Firesmith: "Using Quality Models to Engineer Quality Requirements", in Journal of Object Technology, vol. 2, no. 5, September-October 2003, pp. 67- 75. http://www.jot.fm/issues/issue_2003_09/column6
5. ISO/IEC 25000:2005 Software Engineering -- Software product Quality Requirements and Evaluation (SQuaRE) -- Guide to SQuaRE
6. Portal ISO25000.com
7. https://es.wikipedia.org/wiki/ISO/IEC_25000
8. https://en.wikipedia.org/wiki/Non-functional_requirement (Contiene una lista muy extensa de requisitos NF, con mucho detalle en la mayor parte de los casos, es una muy buena referencia)
9. <http://www.overti.es/tecnologia/283-normas-y-estandares-para-gestion-de-requisitos>

BIBLIOGRAFÍA PARA AMPLIAR EL TEMA

1. Martín, James. Information Engineering: Book II- Planning and Análisis. Prentice Hall. 1990
2. Gerald Kotonya and Ian Sommerville, Requirements Engineering – Processes and Techniques, Wiley, 1998
3. <https://en.wikipedia.org/wiki/Privacy>